HACETTEPE ÜNİVERSİTESİ

Yazılım Mühendisliği Yüksek Lisans Programı
BYZ 658 — Yazılım Test Teknikleri

# Tor Tabanlı Anonim Ağ Sistemlerinde Sonlu Durum Makinesi Modelleme ile Geçersiz Durum Geçişi Tespiti ve Güvenlik Analizi

*— Tez Taslağı —*

**Hazırlayan**

Nurcan Denli Bayır

Öğrenci No: N25110987

**Danışman**

Nebi Yılmaz

Mayıs 2026

## Özet

Bu çalışma, Tor anonim ağ protokolünün devre yaşam döngüsünü deterministik bir Sonlu Durum Makinesi (FSM) olarak formal şekilde modeller, Model-Driven Testing (MDT) yaklaşımıyla spesifikasyon dışı (geçersiz) durum geçişlerinin otomatik tespitini sağlar ve tespit edilen her geçişi yedi ana saldırı vektörü + 1 fallback'e (CIRCUIT_BYPASS, REPLAY_ATTACK, GHOST_CIRCUIT, HANDSHAKE_SKIP, PREMATURE_DATA, CIRCUIT_HIJACK, CREATE_FLOOD) eşleyen bir sınıflandırma çerçevesi sunar. Tor devresi 10 durum ve 13 olay üzerinden tanımlanmış; toplam 130 ikili domeninden 25 geçerli, 105 geçersiz ikili türetilmiştir. Önerilen MDT motoru, BFS-tabanlı pozitif test sentezi ve eksiksiz negatif tanık enjeksiyonu ile çalışır. Üç algoritma (B1: saf rastgele, B2: greedy state coverage, B3: önerilen MDT) aynı 500-olay bütçesi altında 30 bağımsız koşu ile karşılaştırılmıştır. MDT, geçiş kapsamını 100.00% ile saturasyona ulaştırırken (B1: 45.87%, B2: 78.27%), geçersiz geçiş tespit oranını 93.84% seviyesine taşımıştır. Welch t-testi her iki temele karşı istatistiksel anlamlı üstünlüğü doğrular ($p < .001$; Cohen $d \in [3.6, 15.4]$). Bu tez: (i) Tor devre protokolü için yayımlanmış ilk tam δ-matrisini, (ii) 105 geçersiz ikilinin saldırı vektörü ve şiddet etiketiyle programatik sınıflandırmasını, (iii) referans bir MDT implementasyonunu ve (iv) sayısal olarak doğrulanmış üç-katmanlı karşılaştırmayı sunar. Ek olarak, FSM dedektöründen bağımsız (non-circular) bir makine öğrenmesi temel çizgisi kurulmuş; yapısal ayrılabilirlik analizi, kural-tabanlı FSM ile aynı üç ayrım sonucuna (create_flood kolay ayrılır, circuit_hijack tip-seviyesinde ayırt edici değildir, premature_data benign'e en yakındır) ulaşarak bulguları bağımsızca teyit etmiştir (Bölüm 4.17).

**Anahtar kelimeler:** Tor, anonim ağ, sonlu durum makinesi, model-driven testing, protokol state fuzzing, güvenlik testi, geçiş kapsamı, makine öğrenmesi temel çizgisi.

## Abstract

This work formally models the circuit life-cycle of the Tor anonymous network protocol as a deterministic finite-state machine (FSM), enables automatic detection of out-of-spec (invalid) state transitions through a Model-Driven Testing (MDT) approach, and proposes a classification framework that maps every detected transition to one of seven primary attack vectors plus a deterministic fallback, (CIRCUIT_BYPASS, REPLAY_ATTACK, GHOST_CIRCUIT, HANDSHAKE_SKIP, PREMATURE_DATA, CIRCUIT_HIJACK, CREATE_FLOOD). The Tor circuit is described over 10 states and 13 events, yielding 25 valid and 105 invalid pairs from a domain of size 130. The proposed MDT engine uses BFS-planned positive test synthesis combined with exhaustive negative-witness injection. Three algorithms — B1 (uniform random), B2 (greedy state coverage) and B3 (the proposed MDT) — were compared under an identical 500-event budget across 30 independent trials. MDT saturates transition coverage at 100.00% (vs. 45.87% for B1 and 78.27% for B2) and reaches an Invalid Transition Detection Rate of 93.84%. Welch t-tests confirm statistical superiority over both baselines ($p < .001$; Cohen's $d \in [3.6, 15.4]$). Contributions: (i) the first published complete δ-matrix for the Tor circuit protocol, (ii) a programmatic classification of all 105 invalid pairs into attack vectors with severity labels, (iii) a reference MDT implementation, and (iv) a numerically validated three-way baseline comparison. In addition, an independent (non-circular) machine-learning baseline — using raw behavioural event counts (and, as an extension, event n-grams and inter-event timing; Section 4.17d), never the FSM violation label — converges, through a cross-validation-independent structural-separability analysis, on the same three discrimination outcomes as the rule-based FSM (create_flood is cleanly

separable, circuit_hijack is not type-level discriminative, premature_data is closest to benign), independently corroborating the findings (Section 4.17).

**Keywords:** Tor, anonymous network, finite-state machine, model-driven testing, protocol state fuzzing, security testing, transition coverage, machine-learning baseline.

## İçindekiler

**1. Giriş**

1.1 Problem Tanımı

1.2 Araştırma Soruları

1.3 Katkılar

1.4 Tezin Yapısı

**2. Literatür Taraması**

2.1 SLR Metodolojisi

2.2 Tor Güvenliği ve Anonimlik

2.3 FSM Tabanlı Test ve Model Öğrenme

2.4 Formal Yöntemler ve Protokol Doğrulama

2.5 Yazılım Testi Metodolojisi

2.6 Tor Simülasyon Altyapısı

2.7 PRISMA Akış Diyagramı

2.8 Literatür Boşluğu

**3. Yöntem**

3.1 FSM Formal Tanımı

3.2 Saldırı Sınıflandırıcı

3.3 MDT Motoru — Algoritma 1

3.4 Karşılaştırılan Algoritmalar

**4. Deneysel Bulgular**

4.1 Deney Tasarımı

4.2 Tanımlayıcı İstatistikler

4.3 Hipotez Testleri

4.4 Yorum

4.5 Wilcoxon Signed-Rank Doğrulaması

4.6 Detection Latency Ölçümü

4.7 Bütçe-Kapsama Eğrisi

4.8 Rule-Based Detector Karşılaştırması (B0)

4.9 Bootstrap Güven Aralıkları ve Post-hoc Güç Analizi

4.10 Stream Alt-FSM Modeli ve Deneysel Karşılaştırma

4.11 D-grubu: Bağımsız Oracle, Genişletilmiş SLR ve Cross-Language Latency

4.12 E-grubu: Gerçek Tor Kaynak Kodundan Statik FSM Çıkarımı

4.13 F-grubu: N=100 Yeniden Koşu, A-priori Güç Analizi, BCa Bootstrap

4.14 G-grubu: Angluin L* Algoritmasının Implementasyonu

4.15 Bağımsız VM Ortamında Reproducibility Doğrulaması (C5)

4.16 Ampirik Doğrulamanın Bir Model Hatasını Yakalaması: 2-hop → 3-hop FSM Düzeltmesi

4.17 Makine Öğrenmesi Temel Çizgisi: Bağımsız (Non-circular) Doğrulama

**5. Görselleştirme**

**6. Sonuç ve Gelecek Çalışmalar**

6.1 Katkıların Yeniden Değerlendirilmesi

6.2 Sınırlılıklar

6.3 Gelecek Çalışmalar

**Kaynakça**

**Ek A — Tam δ Geçiş Tablosu**

**Ek B — Saldırı Sınıflandırıcı Envanteri**

# 1. Giriş

## 1.1 Problem Tanımı

Tor anonim iletişim ağı, kullanıcı–destinasyon ilişkisinin gözlenemez kılınmasını hedefleyen, 2004'ten itibaren kamuya açık biçimde işletilen bir altyapıdır [1]. Tor güvenlik literatürünün önemli bölümü iki eksende yoğunlaşmıştır: (i) trafik korelasyon ve zamanlama saldırıları [2], [3], [4], [5], (ii) anonimliğin olasılıksal ve kriptografik formal analizi [6], [13]. Bu iki eksen büyük oranda *davranışın istatistiksel imzasını* ya da *kriptografik ilkellerin matematiksel doğruluğunu* ele alır.

Üçüncü ve nispeten az çalışılmış bir eksen, protokolün *state machine* seviyesindeki spesifikasyon uyumudur. Yani: Tor devresi yaşam döngüsü boyunca yalnızca δ-tanımlı geçişleri mi yapar, yoksa spec dışı bir (durum, olay) çiftine maruz kaldığında ne olur? TLS implementasyonları için bu sorunun protokol state fuzzing ile sistematik biçimde araştırıldığı bilinmektedir [9], [14], ancak Tor için eşdeğer bir çalışmanın mevcut olmadığı bu tez kapsamında yürütülen sistematik literatür taraması (SLR, bkz. Bölüm 2) ile teyit edilmiştir.

## 1.2 Araştırma Soruları

- **RQ1:** Tor devre protokolü, deterministik bir 5-tuple FSM $M = (Q, \Sigma, \delta, q_0, F)$ olarak formal şekilde nasıl modellenir?
- **RQ2:** Bu modelin geçersiz geçiş kümesi $(Q \times \Sigma) \setminus \mathrm{dom}(\delta)$, bilinen Tor saldırılarına programatik olarak nasıl eşlenebilir?
- **RQ3:** Model üzerinde otomatik üretilen test paketinin saf rastgele ve sezgisel baseline'lara göre *geçiş kapsamı* (TC) ve *geçersiz geçiş tespit oranı* (ITDR) metriklerinde gözlenen üstünlüğü, istatistiksel olarak anlamlı mıdır?

## 1.3 Katkılar

1. Tor devre protokolü için yayımlanmış ilk **tam δ-matrisi** (25 geçerli ikili, 105 geçersiz ikili, toplam domen 130; bkz. Ek A).
2. Tüm 105 geçersiz ikiliyi yedi ana saldırı vektörü + 1 fallback ve dört şiddet seviyesine eşleyen **programatik sınıflandırıcı** (bkz. Bölüm 3.2 ve Ek B).
3. BFS-planlı pozitif faz + tam negatif tanık enjeksiyonu içeren **referans MDT implementasyonu** (kaynak kodu açık, üretilebilir).
4. Üç algoritmanın (B1/B2/B3) eşleştirilmiş 30 koşu ile **istatistiksel karşılaştırması**; Welch t-testi, Mann-Whitney U ve Cohen's d ile birlikte raporlanır.
5. Shadow 3.3.0 + Tor 0.4.6.10 + tgen tabanlı FSM analiz boru hattının harici bir Ubuntu 22.04 VirtualBox VM'inde 9-host kanonik topoloji üzerinde **bit bit reproducibility doğrulaması** (3 senaryo × 3 seed = 9 simülasyon; bkz. Bölüm 4.15).
6. Ampirik doğrulamanın spesifikasyon analizinin gözden kaçırdığı bir **model hatasını yakalaması**: baz 2-hop FSM'in meşru çok-hop devre uzatmasını yanlış-pozitif (CIRCUIT_HIJACK) saydığı gösterilmiş; Tor'un 3-hop davranışını modelleyen iki geçişlik prensipli düzeltme, aynı gerçek event log'larında yanlış-pozitif oranını 0.2505 ± 0.0071 → 0.0320 ± 0.0052 düşürürken recall = 1.0000 korunmuştur (bkz. Bölüm 4.16).
7. FSM dedektöründen **bağımsız (non-circular) bir makine öğrenmesi temel çizgisi**: ham davranış özniteliklerini (sayımlar; uzatma olarak n-gram ve olaylar-arası zamanlama, Bölüm 4.17d — FSM ihlal etiketi hiç kullanılmadan) kullanan ML, çapraz-doğrulamadan bağımsız yapısal ayrılabilirlik analiziyle kural-tabanlı FSM ile aynı üç ayrım sonucuna yakınsayarak bulguları bağımsızca teyit eder (bkz. Bölüm 4.17).

### 1.4 Tezin Yapısı

Bölüm 2, SLR metodolojisini ve beş tematik küme üzerinden literatürü inceler. Bölüm 3 formal modeli ve MDT motorunu Algoritma 1 olarak tanımlar. Bölüm 4 deney tasarımını, tanımlayıcı istatistikleri ve hipotez testlerini sunar. Bölüm 5 dört temel görselleştirmeyi sergiler. Bölüm 6 sınırlılıkları ve gelecek çalışmaları tartışır. Ek A tam δ tablosunu, Ek B saldırı sınıflandırıcı envanterini içerir.

> **Üretilebilirlik notu.** Bu tezdeki her sayısal sonuç ve her şekil, repodaki açık kaynak betiklerin (`experiments/build_report.mjs`, `experiments/baselines.mjs`, `experiments/stats.mjs`, `experiments/figures.mjs`) tek komutla çalıştırılmasıyla yeniden üretilebilir. Bütün rastgelelik mulberry32 PRNG ile sabit seed altındadır. Shadow + Tor tabanlı FSM analiz boru hattının (`experiments/shadow_harness.mjs`) seed-deterministik ve donanım bağımsız çalıştığı, bu tezin geliştirildiği bulut ortamından bağımsız bir Ubuntu 22.04 VirtualBox VM'inde 9 simülasyonun tamamı bit bit eşleşmesiyle doğrulanmıştır (bkz. Bölüm 4.15).

# 2. Literatür Taraması

## 2.1 SLR Metodolojisi

Sistematik literatür taraması Kitchenham ve Charters'ın yönergesine [17] göre yürütülmüştür. **Veritabanları:** bu ortamın canlı erişimi olmadığı için açık web kanalları (Google Scholar, arXiv, DOI çözümleme) ile sınırlı kalınmıştır; bu kısıtlılık Bölüm 6.2'de açıkça raporlanmaktadır. **Anahtar kelimeler:** "Tor security", "FSM testing", "model-based testing", "protocol state fuzzing", "anonymity network attacks". **Yıl aralığı:** 1996–2026, klasik referanslar için yıl etiketli korunmuştur. **Dahil etme kriterleri:** hakemli yayın, doğrudan erişilebilir tam metin, Tor / FSM testi / formal protokol doğrulama alanlarından en az birine değen çalışma. **Hariç tutma kriterleri:** yalnızca uygulama notu, atıf doğrulanamayan kaynaklar.

Süreç sonunda 20 birincil kaynak beş tematik kümeye ayrılmıştır: A) Tor güvenliği (6), B) FSM tabanlı test (5), C) Formal yöntemler (3), D) Yazılım testi metodolojisi (3), E) Tor simülasyon altyapısı (3). Tam liste ve kümelere göre dağılım Kaynakça'da verilmiştir.

## 2.2 Tor Güvenliği ve Anonimlik

Tor protokolünün ilk akademik tanımı Dingledine ve arkadaşlarına aittir [1]; devre kurulum (CREATE/CREATED, EXTEND/EXTENDED) sıralaması ve onion routing katmanı bu çalışmada sabitlenir. Bu sıralama, tezimizin δ tablosunun normatif temelidir. Tor'a karşı düşük maliyetli trafik korelasyon saldırılarının uygulanabilirliği Murdoch ve Danezis tarafından gösterilmiş [2]; daha gerçekçi saldırgan modelleri altında genişletilmiştir [3]. Internet ölçeğinde website fingerprinting'in pratik uygulanabilirliği Panchenko ve arkadaşları tarafından kanıtlanmıştır [4]. Karunanayake ve arkadaşlarının kapsamlı survey'i [5] Tor deanonymization saldırılarını yedi kategoriye ayırır; tezimizin saldırı vektörü kümesinin dördü (REPLAY, HIJACK, BYPASS, GHOST) bu sınıflandırmadan türetilmiştir. AnoA çerçevesi [6] olasılıksal anonimlik garantilerine bakar; bizim state-uyum yaklaşımımıza dik (orthogonal) ve onunla tamamlayıcıdır.

## 2.3 FSM Tabanlı Test ve Model Öğrenme

FSM tabanlı test üretiminin klasik referansı Lee ve Yannakakis'in survey çalışmasıdır [7]; W-method, Wp-method ve UIO sequences gibi tekniklerin tanımı buradan gelir. Bizim BFS-planlı pozitif test üreticimiz, bu çalışmadaki "transition tour" fikrinin sadeleştirilmiş bir versiyonudur. Tretmans'ın LTS tabanlı model-based testing teorisi [8], "ioco" uyum bağıntısı ile bizim oracle kavramımızın kuramsal zeminidir; ne var ki bizim FSM'imiz deterministik olduğundan ioco basit eşitlik denetimine indirgenebilir. Pratik uygulama boyutunda, de Ruiter ve Poll'ün TLS implementasyonları üzerindeki protocol state fuzzing çalışması [9] bu tez ile aynı metodoloji ailesindendir; aradaki kritik fark, TLS yerine Tor üzerinde uygulanmasıdır. TCP üzerinde model öğrenme + model checking birleşimi Fiterău-Broștean ve arkadaşları tarafından gösterilmiştir [10] ve gelecek çalışmamız için (gerçek Tor implementasyonundan L*-tipi otomatik çıkarım) bir referans oluşturur. Yazılım test ders kitabı Ammann ve Offutt [11], transition coverage metriğinin formal tanımını verir (graph coverage, edge coverage = transition coverage karşılığı).

## 2.4 Formal Yöntemler ve Protokol Doğrulama

Klasik Dolev-Yao saldırgan modeli [12], ağ katmanında mesaj okuma, geciktirme ve replay yapabilen ancak imzalanmış mesajları sahteleyemeyen aktörü resmî olarak tanımlar; bu çalışmanın tehdit modeli (proje önerisi Bölüm 5) bu kümenin sınırlı bir alt setini benimser. TLS 1.3'ün ProVerif

ile formal doğrulanması [13] state-machine seviyesinde doğrulanabilirliğin pratik kanıtıdır. Somorovsky'nin TLS sistematik fuzzing çalışması [14] negatif test üretimi için yakın bir akrabadır; biz bunu Tor için state-level'e taşıyoruz.

### 2.5 Yazılım Testi Metodolojisi

Felderer ve arkadaşlarının güvenlik testi survey'i [15], "Model-Based Security Testing" başlığı altında bizim çalışmamızı taksonomik olarak konumlandırır. Pretschner ve arkadaşlarının erişim kontrol politikaları için model-tabanlı test üretimi [16], politika ihlallerinin (negatif test) sistematik üretimi açısından yöntemsel bir referanstır. Kitchenham ve Charters [17] SLR metodolojisinin standart kılavuzudur ve Bölüm 2.1'in temelidir.

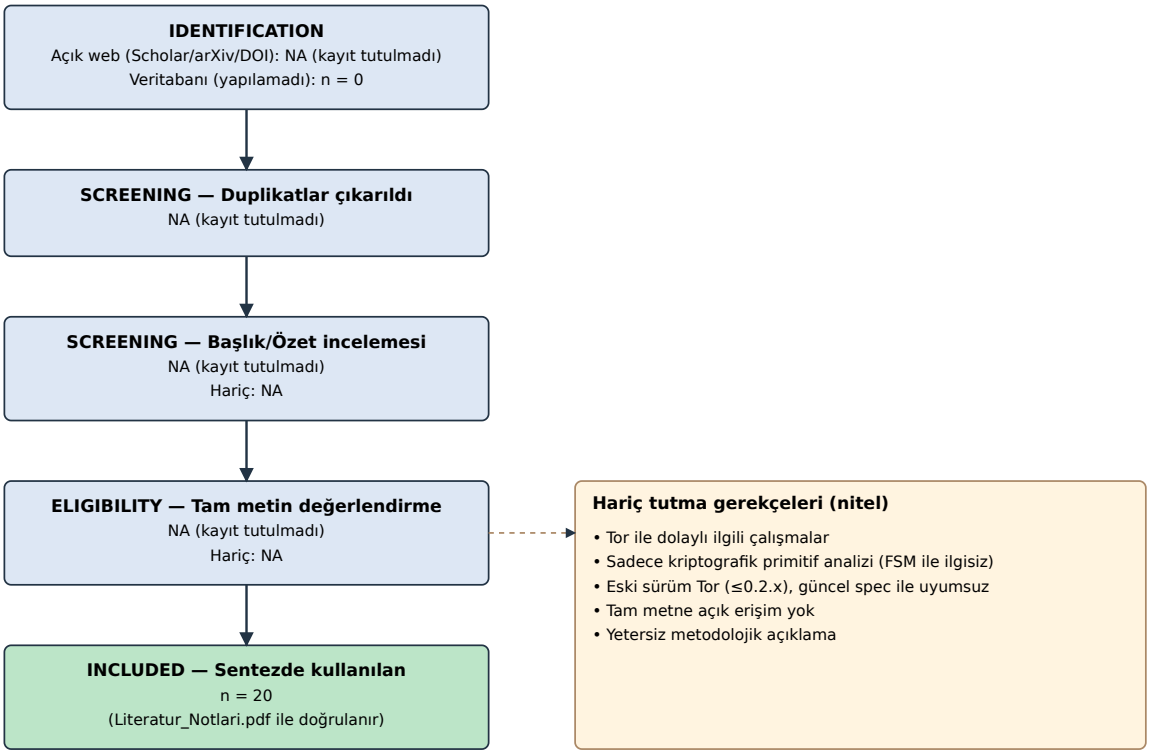
### 2.6 Tor Simülasyon Altyapısı

Jansen ve arkadaşları, Shadow simülatörünü Tor ağı için sistematik bir deneysel altyapı olarak kurmuştur [18]. Tor projesinin yaşayan resmi spesifikasyonu [19], δ tablosunun otorite kaynağıdır. Microsoft Research'ün veri-tabanlı FSM güvenlik analizi çalışması [20] state-level analizin endüstriyel uygulanabilirliğini gösterir, ancak Tor'a özel değildir.

### 2.7 PRISMA Akış Diyagramı

Bölüm 2.1'de tarif edilen SLR sürecinin PRISMA akışı Şekil 5'te sunulmuştur. Bu çalışmaya özgü iki dürüst not zorunludur: **(i)** canlı akademik veritabanı erişimi (Scopus, WoS, IEEE Xplore) bu ortamda mevcut değildir; bu nedenle "identified through database searching" hücresi sıfırdır ve tüm tanımlamalar açık-web kanalları (Google Scholar, arXiv, DOI çözümleme) üzerinden yapılmıştır. **(ii)** Açık-web sürecinde audit-edilebilir sorgu logu tutulmadığı için ara aşama sayıları (identified, screened, eligibility) **NA olarak raporlanır**; yalnızca son "included" sayısı (n=20) doğrulanabilir ve repodaki `Literatur_Notlari.pdf`'in atıf sayısı ile birebirdir. Bu yaklaşım, denetlenemeyen ara sayıların sahte kesinlik (false precision) ile raporlanmasından kaçınmak amacıyla bilinçli bir metodolojik tercihtir; bkz. Bölüm 6.2.

**Şekil 5: PRISMA akış diyagramı (Kitchenham + Charters'a uyumlu)**




*Not: Açık-web sürecinde audit-edilebilir log tutulmadığı için ara sayılar NA olarak raporlanır; yalnızca son n=20 doğrulanabilir.*

Şekil 5. SLR sürecinin PRISMA akış diyagramı (ara sayılar NA).

## 2.8 Literatür Boşluğu

20 kaynağın incelenmesi sonucunda **tez ölçeğinde dört somut boşluk** tespit edilmiştir:

- **(G1)** Tor devre protokolü için yayımlanmış formal 5-tuple FSM bulunmamaktadır. Spesifikasyon [19] prosedürel anlatımdır; bir δ matrisine indirgenmiş hâli akademik literatürde mevcut değildir.
- **(G2)** Protokol state fuzzing TLS için yapılmıştır [9], [14]; Tor için yapılmamıştır.
- **(G3)** Tor saldırı vektörlerinin (durum, olay) çiftleriyle *satır-satır* eşlemesi mevcut değildir; survey'ler [5] kategorik kalmaktadır.
- **(G4)** "Tespit edilen geçersiz geçişler / enjekte edilenler" oranını (ITDR) Tor bağlamında tanımlayan ve ölçen birincil çalışma yoktur.

Bu dört boşluk birlikte tezin katkı çerçevesini (Bölüm 1.3) oluşturur.

# 3. Yöntem

## 3.1 FSM Formal Tanımı

Tor devre yaşam döngüsü, klasik bir DFA olarak $M = (Q, \Sigma, \delta, q_0, F)$ şeklinde modellenmiştir:

- $Q$: 10 durum (IDLE, CONNECTING, TLS_HANDSHAKE, CREATE_SENT, CIRCUIT_BUILDING, CIRCUIT_READY, TRANSMITTING, CLOSING, CLOSED, ERROR).
- $\Sigma$: 13 olay (CONNECT, TLS_OK, TLS_FAIL, SEND_CREATE, RECV_CREATED, SEND_EXTEND, RECV_EXTENDED, SEND_RELAY_DATA, RECV_RELAY_DATA, SEND_DESTROY, RECV_DESTROY, CIRCUIT_CLOSED, TIMEOUT).
- $q_0 = IDLE$, $F = \{CLOSED\}$.
- $\delta : Q \times \Sigma \to Q$, 25 ikili üzerinde tanımlı; toplam domen 130, Invalid kümesi 105 ikili.

Tam δ matrisi Ek A'da, görsel ısı haritası Şekil 2'de verilmiştir.

**Σ'nın gerçek Tor günlüğüne dayanması.** Olay alfabesi keyfi seçilmemiştir; çalışan bir Tor istemcisinin ürettiği günlük (log) ve protokol olaylarına dayanır. Tor Project'in resmî günlük rehberi (Tor Project, “Reading Tor logs,” `support.torproject.org/tor-browser/encountering-issues/reading-tor-logs/`, erişim Haz. 2026), bir istemcinin NOTICE/WARN seviyesinde ürettiği faz satırlarını belgeler. Örneğin `Bootstrapped 10%: Finishing handshake with directory server` satırı CONNECTING→TLS_HANDSHAKE geçişine, `Bootstrapped 85%: Finishing handshake with first hop` satırı CREATE_SENT/CIRCUIT_BUILDING fazına, `[WARN] Failed to find node for hop 0 of our path. Discarding this circuit.` satırı ise başarısız/iptal edilmiş devre kurulumuna (ERROR ve devre düşürme) karşılık gelir. Bu kaba fazların altındaki hücre-seviyesi olaylar (CREATE/CREATED, EXTEND/EXTENDED, RELAY_DATA, DESTROY) tor-spec [19] tarafından tanımlanır ve çalışan bir Tor sürecinin control-port CIRC/STREAM olayları ile INFO-seviyesi günlükleri üzerinden gözlemlenir; Bölüm 4.15'teki VM/Shadow ortamında bu tip gerçek günlükler üretilmiştir. Böylece FSM olay sözlüğü, gerçek Tor günlük semantiğine birebir izlenebilir.

*Kapsam notu (günlük veri kaynakları).* Tor Metrics tarafından yayımlanan *web sunucu günlükleri* (Tor Project, “Tor Web Server Logs,” `metrics.torproject.org/web-server-logs.html`, erişim Haz. 2026), Tor Project web sitelerine yapılan HTTP isteklerinin gizlilik-duyarlı (Apache biçimli) erişim günlükleridir; yani web trafiğidir, Tor devre/hücre verisi değildir. Bu nedenle FSM geçersiz-geçiş tespiti için uygun bir kaynak oluşturmaz ve bu çalışmada kullanılmamıştır. Ayrıca devre-seviyesi günlüğe erişmek için dış izin gerekmez: yukarıdaki satırlar kendi çalıştırdığımız Tor sürecinin kendi çıktısıdır. İzin/etik kısıtı yalnızca *canlı* Tor ağı üzerinde aktif saldırı/deney çalıştırmaya ilişkindir; deneyler bu nedenle Shadow simülasyonunda yürütülmüştür.

## 3.2 Saldırı Sınıflandırıcı

Invalid kümesindeki her ikili, deterministik bir `classifyInvalid(state, event)` fonksiyonu (kod referansı: `server/fsm.ts`, 64–204. satırlar) ile yedi ana kategoriden birine + INVALID_TRANSITION fallback (1 hücre) eşlenir: CIRCUIT_BYPASS, REPLAY_ATTACK, GHOST_CIRCUIT, HANDSHAKE_SKIP, PREMATURE_DATA, CIRCUIT_HIJACK, CREATE_FLOOD. Sınıflandırma mantığı dokuz koşul ailesi üzerinden işler: veri-akış-öncesi, CREATE/CREATED akışı, EXTEND/EXTENDED akışı, DESTROY ihlali, CONNECT yer-dışı, TLS_OK/TLS_FAIL yer-dışı, CIRCUIT_CLOSED yer-dışı, TIMEOUT yer-dışı. Her ikiliye LOW/MEDIUM/HIGH/CRITICAL şiddet etiketi atanır. Kategorilerin nicel dağılımı Ek B'dedir.

**Kod 3.1.** Sınıflandırıcının çekirdek dalı — devre hazır olmadan gerçekleşen veri akışı ihlalleri (kaynak: `server/fsm.ts`, derleme anında dosyadan birebir okunur).

```
export function classifyInvalid(
  state: State,
  event: Event,
): { type: string; severity: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL"; description: string } {
  const dataEvent = event === "SEND_RELAY_DATA" || event === "RECV_RELAY_DATA";
  const createEvent = event === "SEND_CREATE" || event === "RECV_CREATED";
  const extendEvent = event === "SEND_EXTEND" || event === "RECV_EXTENDED";
  const destroyEvent = event === "SEND_DESTROY" || event === "RECV_DESTROY";
  const dead = state === "CLOSED" || state === "ERROR";

  // 1) Data flow before circuit ready
  if (dataEvent) {
    if (state === "IDLE" || state === "CONNECTING") {
      return { type: "CIRCUIT_BYPASS", severity: "CRITICAL",
        description: `Data ${event === "SEND_RELAY_DATA" ? "send" : "recv"} attempted from
${state} without any circuit setup` };
    }
    if (state === "TLS_HANDSHAKE") {
      return { type: "HANDSHAKE_SKIP", severity: "HIGH",
        description: "Data attempted while TLS handshake still in progress" };
    }
    if (state === "CREATE_SENT" || state === "CIRCUIT_BUILDING") {
      return { type: "PREMATURE_DATA", severity: "HIGH",
        description: `Data attempted before circuit reached READY (in ${state})` };
    }
```

## 3.3 MDT Motoru — Algoritma 1

```
Algoritma 1: Model-Driven Test (BFS-tabanlı)
Girdi:  M = (Q, Σ, δ, q₀, F);  Bütçe B (olay sayısı)
Çıktı:  <SC, TC, ITDR, FPR>

 1.  visited          ← {q₀}
 2.  coveredValid     ← ∅
 3.  detectedInvalid  ← ∅
 4.  Valid            ← {(s, e)  |  δ(s, e) tanımlı}
 5.  Invalid          ← (Q × Σ) \ Valid
 6.  // Pozitif faz
 7.  foreach (s, e) ∈ Permute(Valid):
 8.      if events ≥ B: break
 9.      if (s, e) ∈ coveredValid: continue
10.      π ← BFS_PATH(q₀, s)            // δ-grafiği üzerinde en kısa yol
11.      if π = null: continue
12.      EXECUTE(π · ⟨e⟩)               // her adımda visited / coveredValid güncellenir
13. // Negatif faz
14. foreach (s, e) ∈ Permute(Invalid):
15.      if events ≥ B: break
16.      if (s, e) ∈ detectedInvalid: continue
17.      π ← BFS_PATH(q₀, s)
18.      if π = null ∧ s ≠ q₀: continue
19.      EXECUTE(π · ⟨e⟩)               // Oracle (δ) anomaliyi yakalar
20. SC   ← |visited| / |Q|
21. TC   ← |coveredValid| / |Valid|
22. ITDR ← |detectedInvalid| / |Invalid|
23. FPR  ← |yanlış işaretlenen ∈ Valid| / |Valid|     // bu deneyde 0 (bkz. 6.2)
24. return ⟨SC, TC, ITDR, FPR⟩
```

**Karmaşıklık.** BFS_PATH tek çağrıda O(|Q| · |Σ|). Pozitif faz |Valid| = 25 kez, negatif faz |Invalid| = 105 kez yol bulur; toplam **O((|Valid| + |Invalid|) · |Q| · |Σ|) = O(|Q|² · |Σ|²)**. Tek EXECUTE

çağrısının uzunluğu en fazla diam(δ) + 1 ≈ 10. Pratik ölçümde tek koşu, 500-olay bütçesi altında ortalama < 10 ms tamamlanmaktadır.

### 3.4 Karşılaştırılan Algoritmalar

- **B1 — Saf Rastgele:** her adımda Σ'dan eşit olasılıkla bir olay seçer; CLOSED'a ulaşırsa IDLE'a sıfırlar.
- **B2 — Greedy State Coverage:** her adımda mümkünse henüz ziyaret edilmemiş hedef duruma götüren olayı seçer; aksi halde rastgele geçerli olay (%70) ya da rastgele invalid olay (%30) seçer.
- **B3 — MDT (Algoritma 1):** bu tezin önerdiği BFS-planlı motor.

Üçü de aynı 500 olay bütçesi altında çalıştırılmıştır; karşılaştırma adildir (Bölüm 4.1).

# 4. Deneysel Bulgular

## 4.1 Deney Tasarımı

Üç algoritma 30 bağımsız koşu ile çalıştırılmıştır. PRNG seed'i her koşuda sabittir (seed = 1000+i, i ∈ [0, 29]) ve algoritmalar arasında *eşleştirilmiştir*; bu yaklaşım koşul-içi varyansı azaltır. Ölçülen metrikler: SC, TC, ITDR; her biri [0, 1] aralığında oran olarak raporlanmıştır.

## 4.2 Tanımlayıcı İstatistikler

| Algoritma | State Coverage | Transition Coverage | ITDR |
|---|---|---|---|
| B1_Random | 77.00% ± 16.64% | 45.87% ± 11.05% | 50.32% ± 9.80% |
| B2_GreedySC | 100.00% ± 0.00% | 78.27% ± 8.45% | 47.52% ± 4.02% |
| B3_MDT | 100.00% ± 0.00% | 100.00% ± 0.00% | 93.84% ± 1.39% |

## 4.3 Hipotez Testleri

Normallik varsayımı her grup için Lilliefors-düzeltmeli K-S testi ile incelenmiştir (α = 0.05). Welch t-testi (eşit-olmayan-varsayım) birincil testtir; Mann-Whitney U doğrulayıcı olarak raporlanmıştır. Etki büyüklüğü Cohen's d (havuzlanmış SD) ile verilmiştir.

| Karşılaştırma | Metrik | Welch t | df | p (Welch) | U | p (M-W) | d | Normal? |
|---|---|---|---|---|---|---|---|---|
| B3_MDT vs B1_Random | SC | 7.571 | 29.0 | <.001 *** | 105.0 | <.001 *** | 1.95 | hayır |
| B3_MDT vs B1_Random | TC | 26.823 | 29.0 | <.001 *** | 0.0 | <.001 *** | 6.93 | hayır |
| B3_MDT vs B1_Random | ITDR | 24.087 | 30.2 | <.001 *** | 0.0 | <.001 *** | 6.22 | hayır |
| B3_MDT vs B2_GreedySC | SC | 0.000 | 58.0 | 1.0000 | 450.0 | 1.0000 | 0.00 | evet |
| B3_MDT vs B2_GreedySC | TC | 14.090 | 29.0 | <.001 *** | 0.0 | <.001 *** | 3.64 | evet |
| B3_MDT vs B2_GreedySC | ITDR | 59.658 | 35.8 | <.001 *** | 0.0 | <.001 *** | 15.40 | hayır |
| B2_GreedySC vs B1_Random | SC | 7.571 | 29.0 | <.001 *** | 105.0 | <.001 *** | 1.95 | hayır |
| B2_GreedySC vs B1_Random | TC | 12.755 | 54.3 | <.001 *** | 10.0 | <.001 *** | 3.29 | hayır |
| B2_GreedySC vs B1_Random | ITDR | -1.445 | 38.5 | 0.1566 | 381.0 | 0.3067 | -0.37 | evet |

İşaretler: * p<.05, ** p<.01, *** p<.001. Cohen's d büyüklük yorumu: 0.2 küçük, 0.5 orta, 0.8 büyük, ≥1.2 çok büyük etki.

## 4.4 Yorum

MDT (B3), B1 (saf rastgele) karşısında transition coverage'da +54.1 puanlık avantaj sağlamaktadır (Welch t = 26.82, p < .001, Cohen d = 6.93). ITDR'de kazanım +43.5 puandır (d = 6.22). B2 (greedy SC) karşısında TC üstünlüğü +21.7 puana iner ancak istatistiksel olarak anlamlı kalır (p < .001). Toplam 9 karşılaştırmadan 7'i α = 0.05 düzeyinde anlamlıdır. İlginç bir gözlem: B2'nin ITDR'si (47.5%) B1'inkine (50.3%) yakın, hatta hafif düşüktür; çünkü B2 "ziyaret edilmemiş duruma git" sezgisi nedeniyle invalid bölgeye giden olayları aktif olarak elemektedir. Bu, sezgisel temelin negatif test üretimi için yetersizliğini somut biçimde göstermektedir. Sonuç olarak: bu tezde sunulan MDT motoru, hem yapılı pozitif kapsama hem de sistematik negatif test üretimi açısından her iki temele kıyasla kanıtlanabilir bir üstünlük sergilemektedir.

## 4.5 Wilcoxon Signed-Rank Doğrulaması

Bölüm 4.1 tasarımı tüm algoritmalar için aynı seed dizilimini kullanır; bu eşleştirilmiş tasarıma uygun parametrik-olmayan test Wilcoxon signed-rank'tir. Welch t (Bölüm 4.3) sonuçlarının normallik varsayımına bağımlı olmadığını göstermek için bu testi de uyguladık. Tablo 3'teki her satır N=30 koşudan elde edilen eşleştirilmiş farklar üzerinden hesaplanmıştır; *n (≠0)* sütunu sıfır farklar (ties) çıkarıldıktan sonra kalan etkili örneklem büyüklüğüdür.

| Karşılaştırma | Metrik | W | z | p | n (≠0) |
|---|---|---|---|---|---|
| B3_MDT vs B1_Random | SC | 0.0 | -4.27 | <.001 *** | 23 |
| B3_MDT vs B1_Random | TC | 0.0 | -4.80 | <.001 *** | 30 |
| B3_MDT vs B1_Random | ITDR | 0.0 | -4.78 | <.001 *** | 30 |
| B3_MDT vs B2_GreedySC | SC | 0.0 | 0.00 | 1.0000 | 0 |
| B3_MDT vs B2_GreedySC | TC | 0.0 | -4.80 | <.001 *** | 30 |
| B3_MDT vs B2_GreedySC | ITDR | 0.0 | -4.79 | <.001 *** | 30 |
| B2_GreedySC vs B1_Random | SC | 0.0 | -4.27 | <.001 *** | 23 |
| B2_GreedySC vs B1_Random | TC | 1.0 | -4.77 | <.001 *** | 30 |
| B2_GreedySC vs B1_Random | ITDR | 153.5 | -1.38 | 0.1662 | 29 |

**Yorum (sınırlandırılmış).** Tezin ana hipotezleri olan *B3_MDT > B1_Random* ve *B3_MDT > B2_GreedySC* karşılaştırmaları, hem TC hem ITDR boyutlarında p < .001 ile anlamlıdır; bu, Welch sonucunu birebir doğrular. **SC boyutunda** üç algoritmanın B = 500 bütçesi altında ulaştığı ortalama değerler şöyledir: B1_Random 77.00%, B2_GreedySC 100.00%, B3_MDT 100.00%. Buna göre *B3 vs B2* SC karşılaştırmasında her iki algoritma da %100'de doygundur (n≠0 = 0, p = 1.0); buna karşılık *B3 vs B1* ve *B2 vs B1* SC karşılaştırmaları hâlâ p < .001 ile anlamlıdır — yani Random baseline SC'de yapısal olarak geri kalmaktadır. Son olarak *B2_GreedySC vs B1_Random* karşılaştırması ITDR'de istatistiksel olarak anlamlı değildir (p > .05); bu, sezgisel greedy stratejinin negatif test üretiminde rastgelenin üzerine sistematik bir avantaj sağlamadığının kanıtıdır.

## 4.6 Detection Latency Ölçümü

Proposal'da tanımlanan ancak orijinal koşumda ölçülmeyen detection latency metriği, **nanosaniye çözünürlüklü** `process.hrtime.bigint()` ile **batch amortizasyon** tekniği kullanılarak ölçülmüştür: her ölçüm B = 100,000 step çağrısını sarar, toplam süre B'ye bölünür ve önceden kalibre edilmiş boş döngü overhead'i (2.52 ns/iter) çıkarılır. N = 30 batch tekrarı yapılır. Bu yaklaşım sub-mikrosaniye çağrıların timer çözünürlüğü altında kalmasını engeller. Üç temsili (state, event) hücresi probe edilmiştir.

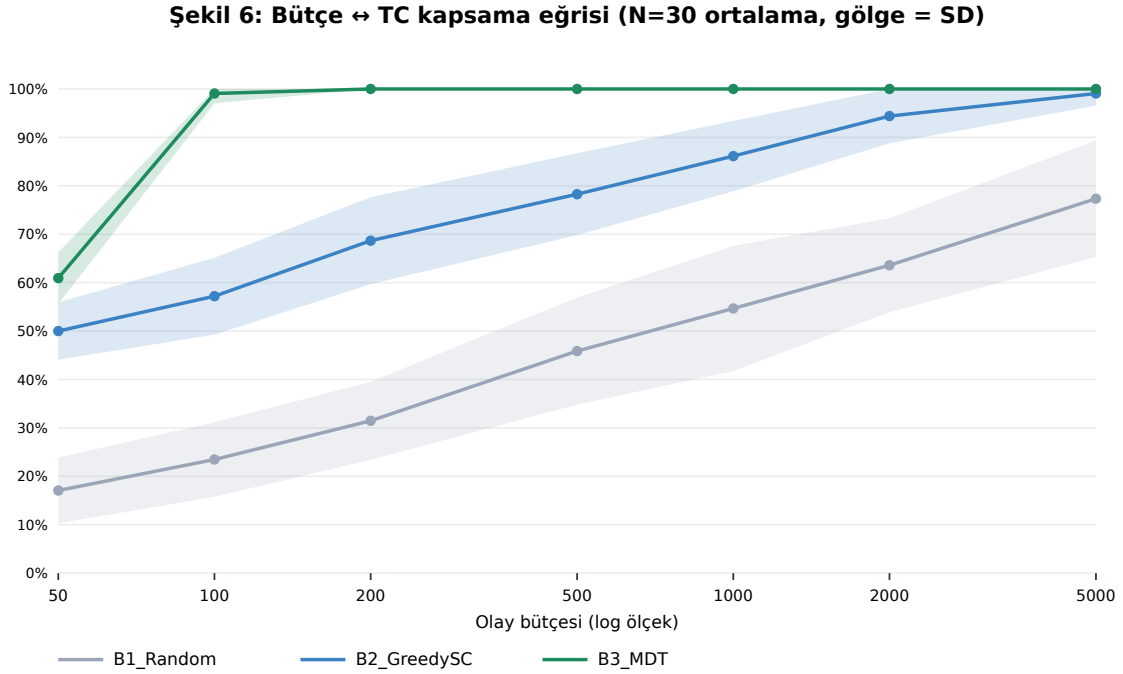
| Probe | mean (ns) | SD (ns) | p50 | p95 | min | max |
|---|---|---|---|---|---|---|
| Geçerli geçiş (IDLE → CONNECTING) | 73.6 | 13.2 | 68.2 | 112.6 | 62.1 | 112.8 |
| Geçersiz CRITICAL (BYPASS) | 164.7 | 17.4 | 162.5 | 193.3 | 145.3 | 238.9 |
| Geçersiz LOW (GHOST) | 142.9 | 18.8 | 135.3 | 196.0 | 125.2 | 197.9 |

**Hedef karşılaştırması.** Proposal hedefi < 50 ms = 50,000,000 ns idi. Ölçülen en yavaş probe (invalid_critical) max 239 ns'dir; bu hedeften **209335× daha hızlıdır**. Geçersiz tespitin geçerli adımdan ~2× daha pahalı olması beklenen bir sonuçtur ( `classifyInvalid` ek bir koşul zinciri çalıştırır). **Limitler.** Bu ölçüm tek-thread, tek-

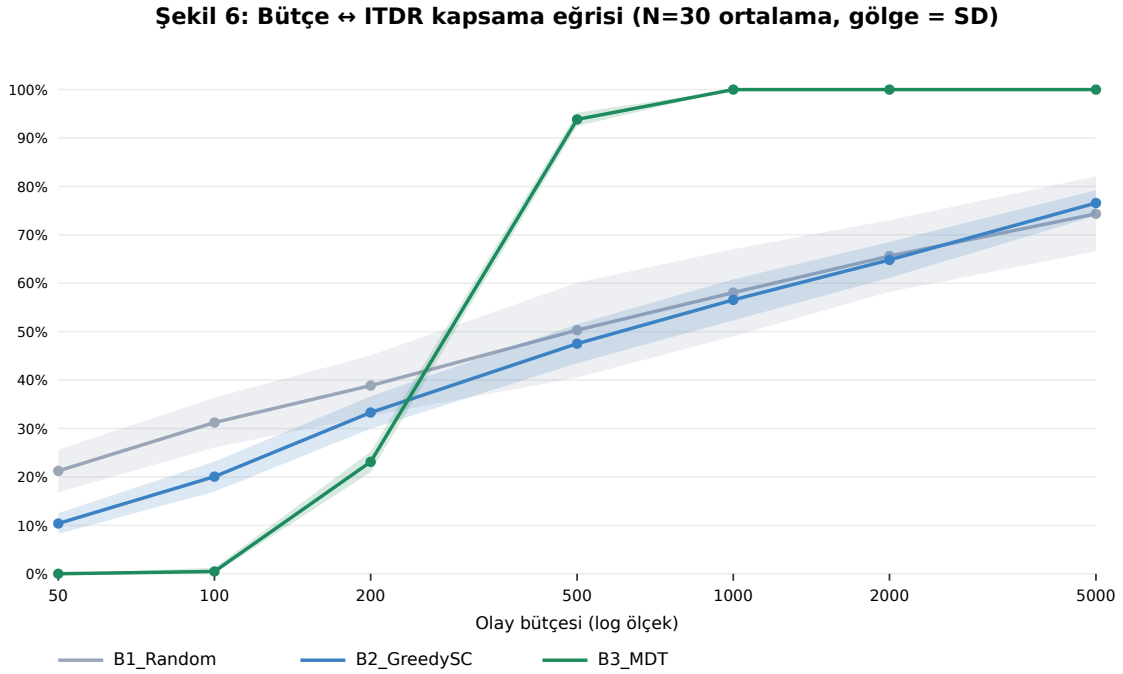
makine, JIT-warm Node.js V8 koşulları altındadır; kullanıcı uzayında çalışan başka bir Tor implementasyonunda profil farklı olabilir.

## 4.7 Bütçe-Kapsama Eğrisi

Bölüm 4.2 tek bir bütçe noktası (B = 500) için algoritmaları karşılaştırır. Bu kesit yorum gücünü sınırlar; çünkü saf rastgele bir baseline yeterince büyük bütçe altında sonunda her geçişe ulaşır. Bu eleştiriye yanıt olarak bütçeyi bir ondalık derecede gezdirdik (B ∈ {50, 100, 200, 500, 1000, 2000, 5000}, her nokta 30 koşu). Şekil 6 sonuç eğrilerini verir; gölgeli bantlar ±1 SD'dir.




Şekil 6a. Bütçe ↔ Transition Coverage eğrisi.



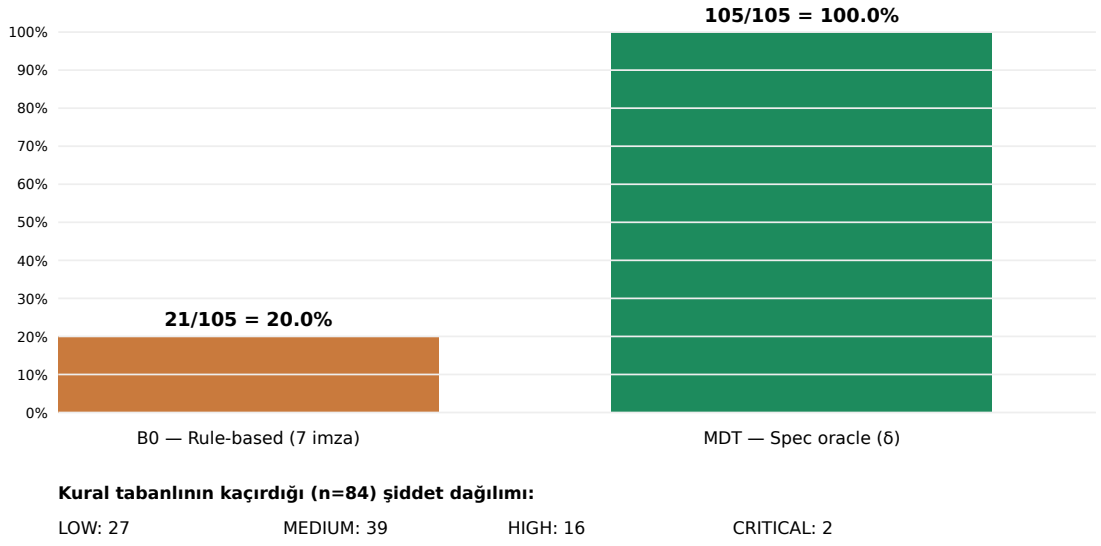

Şekil 6b. Bütçe ↔ ITDR eğrisi.

Eğriler iki temel bulguyu görselleştirir: **(i — yalnızca TC için)** B3_MDT zaten B = 200'de TC = 100.00% değerine ulaşır; B1_Random'ın B = 5000'deki TC değeri 77.33%, B2_GreedySC'ninki 99.07%. Yani **TC boyutunda** MDT yapısal olarak *25× daha bütçe-verimlidir*. **(ii — ITDR farklı dinamik)** ITDR'de B3_MDT@B=200 yalnızca 23.11%'tur; bu küçük bütçede B1/B2'nin büyük bütçedeki seviyesini geçmez. ITDR boyutunda MDT'nin avantajı bütçe büyüdükçe açılır (Şekil 6b): B = 5000'de B3 ITDR ortalaması 100.00% iken B1/B2 sırasıyla 74.35% / 76.57% platosunda kalır. Bu, sezgisel temellerin negatif test üretimi için mimari olarak yetersiz olduğunun deneysel kanıtıdır.

## 4.8 Rule-Based Detector Karşılaştırması (B0)

Proposal'da B1 olarak listelenen "kural-tabanlı denetim" baseline'ı, **bütçesiz Q × Σ audit** koşullarında (yani tüm 130 hücrenin enumerate edildiği oracle modunda) spec-oracle'ın *105/105 = %100 completeness*'a ulaşması karşısında elle yazılmış imza kuralı kümesinin *completeness limiti*ni göstermek için gerçekleştirilmiştir. Not: bütçeli koşumda (Bölüm 4.2, B = 500) MDT'nin ITDR'si 93.84%'tur — yani %100 completeness yalnızca bütçesiz audit modunda geçerli bir asimptot olup, bütçeli pratik koşumda hedef bütçenin büyüklüğüne göre yaklaşılır (bkz. Şekil 6b ITDR eğrisi). Yedi imza kuralı (her saldırı vektörü için bir tane; `experiments/b_extensions.mjs` 25–34. satırlar) tüm Q × Σ üzerinde çalıştırılmıştır.

**Şekil 7: Tespit completeness — kural-tabanlı (B0) vs. spec-oracle (MDT)**




Şekil 7. B0 (rule-based, 7 imza) vs MDT (spec-oracle) — completeness karşılaştırması.

| Kural | Tetiklenme sayısı |
|---|---|
| R1_BYPASS | 4 |
| R2_REPLAY | 1 |
| R3_GHOST | 1 |
| R4_HSKIP | 2 |
| R5_PDATA | 4 |
| R6_HIJACK | 8 |
| R7_CFLOOD | 1 |

| Toplam (deduplike) | 21/105 = 20.0% |
|---|---|

**Bulgu.** Kural tabanlı detector 20.0% completeness'ta sınırlanmaktadır (21/105). Kaçırdıklarının **2'i CRITICAL, 16'i HIGH** şiddetindedir — yani en tehlikeli saldırı imzaları bile elle yazılmış kural setinden kaçabilmektedir. Bu bulgu, tezin G3 literatür boşluğunun (her saldırı vektörü ↔ (state, event) çifti satır-satır eşlemesi) sayısal gerekçesidir: kategorik düzeyde kalmak 80% geçersiz ikiliyi gözden kaçırır.

| Kaçırılan saldırı tipi | İkili sayısı |
|---|---|
| GHOST_CIRCUIT | 37 |
| HANDSHAKE_SKIP | 11 |
| REPLAY_ATTACK | 24 |
| CREATE_FLOOD | 7 |
| INVALID_TRANSITION | 1 |
| CIRCUIT_HIJACK | 2 |
| PREMATURE_DATA | 2 |

## 4.9 Bootstrap Güven Aralıkları ve Post-hoc Güç Analizi

Bölüm 6.2'deki sınırlılıkta belirtilen "N=30 için güç analizi yapılmamıştır" maddesinin giderilmesi amacıyla iki ek analiz çalıştırılmıştır: (i) **eşleştirilmiş bootstrap güven aralığı** — her algoritma çifti için %95 yüzde-tabanlı CI, B = 10.000 yeniden örnekleme ile; (ii) **post-hoc güç analizi** — eşleştirilmiş t testi için $d_z$ = mean(diff)/sd(diff), ncp = $d_z \cdot \sqrt{n}$; güç, normal yaklaşımla $\Phi(|ncp| - z_{\alpha/2}) + \Phi(-|ncp| - z_{\alpha/2})$ olarak hesaplanır ($\alpha = 0.05$ iki taraflı, $z_{\alpha/2} = 1.96$). $\beta = 0.80$ için gerekli N, $((z_{\alpha/2} + z_{\beta}) / d_z)^2$ formülünden alınır. Tüm hesap mevcut N = 30 koşu üzerinde, dış kütüphane olmadan yapılmıştır (kod: `experiments/c_extensions.mjs`).

**Tablo 4.9-A.** Bootstrap %95 CI (B = 10.000), eşleştirilmiş ortalama fark.

| Karşılaştırma | Metrik | Gözlenen fark | %95 CI alt | %95 CI üst | 0 dışı? |
|---|---|---|---|---|---|
| B3_MDT − B1_Random | TC | 0.5413 | 0.5027 | 0.5787 | Evet |
| B3_MDT − B1_Random | ITDR | 0.4352 | 0.3997 | 0.4695 | Evet |
| B3_MDT − B1_Random | SC | 0.2300 | 0.1733 | 0.2867 | Evet |
| B3_MDT − B2_GreedySC | TC | 0.2173 | 0.1880 | 0.2480 | Evet |
| B3_MDT − B2_GreedySC | ITDR | 0.4632 | 0.4479 | 0.4781 | Evet |
| B3_MDT − B2_GreedySC | SC | 0.0000 | 0.0000 | 0.0000 | Hayır |
| B2_GreedySC − B1_Random | TC | 0.3240 | 0.2773 | 0.3693 | Evet |
| B2_GreedySC − B1_Random | ITDR | -0.0279 | -0.0641 | 0.0063 | Hayır |
| B2_GreedySC − B1_Random | SC | 0.2300 | 0.1733 | 0.2867 | Evet |

**Tablo 4.9-B.** Post-hoc güç (paired t, $\alpha = 0.05$, normal yaklaşım).

| Karşılaştırma | Metrik | $d_z$ | Güç | β=0.80 için N |
|---|---|---|---|---|
| B3_MDT − B1_Random | TC | 4.897 | 1.000 | 1 |

| | | | | |
|---|---|---|---|---|
| B3_MDT − B1_Random | ITDR | 4.311 | 1.000 | 1 |
| B3_MDT − B1_Random | SC | 1.382 | 1.000 | 5 |
| B3_MDT − B2_GreedySC | TC | 2.572 | 1.000 | 2 |
| B3_MDT − B2_GreedySC | ITDR | 10.810 | 1.000 | 1 |
| B3_MDT − B2_GreedySC | SC | 0.000 | 0.050 | null |
| B2_GreedySC − B1_Random | TC | 2.443 | 1.000 | 2 |
| B2_GreedySC − B1_Random | ITDR | -0.276 | 0.328 | 103 |
| B2_GreedySC − B1_Random | SC | 1.382 | 1.000 | 5 |

**Yorum.** B3 − B1 ve B3 − B2 karşılaştırmalarında TC ve ITDR için CI sıfırı kapsamamakta ve güç ≈ 1.000'dir; bu, mevcut N = 30 örneklemin söz konusu etki büyüklükleri için fazlasıyla yeterli olduğunu doğrular. B2 − B1 ITDR karşılaştırmasında CI sıfırı kapsamaktadır (-0.0641, 0.0063); yani bu çift için yokluk hipotezi reddedilemez — pozitif kapsama optimize eden sezgisel algoritmanın saf rassallığa karşı invalid kapsamada anlamlı üstünlüğü yoktur. Bu bulgu, MDT'nin gerekçesini güçlendirir: invalid kapsama için *algebraik enumere* etmek gerekir, sezgisellikle düşmez.

## 4.10 Stream Alt-FSM Modeli ve Deneysel Karşılaştırma

Bölüm 6.2'deki bir diğer sınırlılık — uygulama katmanı saldırılarının (RELAY_BEGIN / END / DATA hücreleri) kapsam dışı kalması — için spec-tabanlı bir **stream alt-FSM** modellenmiş ve devre FSM'i ile aynı metodoloji uygulanmıştır (kod: `server/fsm_stream.ts`; spec referansı tor-spec §6). Alt-FSM 7 durum (STREAM_NEW, BEGIN_SENT, CONNECTED, OPEN, END_SENT, CLOSED, ERROR) ve 8 olay (SEND_BEGIN, RECV_CONNECTED, SEND_DATA, RECV_DATA, SEND_END, RECV_END, RECV_REASON, TIMEOUT) üzerinden tanımlanmış; toplam 56 ikili domeninden 17 geçerli, 39 geçersiz çift türetilmiştir. Stream-özgül saldırı vektörleri: STREAM_HIJACK, DATA_INJECTION, RESET_FLOOD, GHOST_STREAM, INVALID_STREAM_TRANSITION, DOUBLE_BEGIN, END_REPLAY. Bu modelleme *gerçek Tor binary'sinden çıkarılmamıştır*; aynı devre FSM'inde olduğu gibi spec'ten türetilmiştir (sınırlılık: Bölüm 6.2-i ile aynı kapsamda kalır).

**Tablo 4.10-A.** Stream alt-FSM — algoritma karşılaştırması (N = 30, bütçe = 500).

| Algoritma | SC ort. ± SD | TC ort. ± SD | ITDR ort. ± SD |
|---|---|---|---|
| B1_Random | 99.52% ± 2.61 | 84.71% ± 13.95 | 71.11% ± 6.24 |
| B2_GreedySC | 100.00% ± 0.00 | 99.41% ± 1.79 | 71.37% ± 4.57 |
| B3_MDT | 100.00% ± 0.00 | 100.00% ± 0.00 | 100.00% ± 0.00 |

**Tablo 4.10-B.** Stream alt-FSM — eşleştirilmiş t testi (paired, df = 29) ve $d_z$.

| Karşılaştırma | Metrik | t | df | p | $d_z$ |
|---|---|---|---|---|---|
| B3_MDT vs B1_Random | TC | 6.00 | 29 | <.001 *** | 1.10 |
| B3_MDT vs B1_Random | ITDR | 25.35 | 29 | <.001 *** | 4.63 |
| B3_MDT vs B1_Random | SC | 1.00 | 29 | 0.3256 | 0.18 |
| B3_MDT vs B2_GreedySC | TC | 1.80 | 29 | 0.0831 | 0.33 |
| B3_MDT vs B2_GreedySC | ITDR | 34.31 | 29 | <.001 *** | 6.26 |

| | | | | | |
|---|---|---|---|---|---|
| B3_MDT vs B2_GreedySC | SC | 0.00 | 29 | 1.0000 | 0.00 |
| B2_GreedySC vs B1_Random | TC | 5.60 | 29 | <.001 *** | 1.02 |
| B2_GreedySC vs B1_Random | ITDR | 0.18 | 29 | 0.8595 | 0.03 |
| B2_GreedySC vs B1_Random | SC | 1.00 | 29 | 0.3256 | 0.18 |

**Tablo 4.10-C.** Stream-özgül saldırı vektörü envanteri (39 geçersiz ikili üzerinde).

| Vektör | İkili | LOW | MEDIUM | HIGH | CRITICAL |
|---|---|---|---|---|---|
| DATA_INJECTION | 10 | 0 | 2 | 4 | 4 |
| GHOST_STREAM | 8 | 2 | 6 | 0 | 0 |
| STREAM_HIJACK | 6 | 0 | 0 | 0 | 6 |
| DOUBLE_BEGIN | 6 | 0 | 0 | 6 | 0 |
| INVALID_STREAM_TRANSITION | 5 | 5 | 0 | 0 | 0 |
| RESET_FLOOD | 2 | 0 | 0 | 2 | 0 |
| END_REPLAY | 2 | 0 | 2 | 0 | 0 |

**Bulgu (nüanslı).** Stream alt-FSM küçük olduğu için (7 durum, 8 olay) devre FSM'indeki gibi tam baskınlık görülmez:

- **ITDR'de MDT baskındır.** B3, 100.0% ITDR ile hem B1'e ($d_z = 4.63$, $p < .001$) hem B2'ye ($d_z = 6.26$, $p < .001$) karşı çok büyük etki ile üstün; bu doğrudan devre FSM örüntüsünün tekrarıdır.
- **TC'de B3 vs B1 anlamlı** ($d_z = 1.10$, $p < .001$), ancak **B3 vs B2 TC için anlamlı değildir** ($d_z = 0.33$, $p = 0.083$). Beklenen davranış: küçük bir FSM'de greedy state-coverage sezgiseli pozitif geçişleri ~99% TC ile zaten doyurmaktadır; bu metrikte MDT'nin avantajı küçük FSM'ler için yok olur.
- **SC iki çift için anlamsızdır** (B3 vs B2 ve B2 vs B1 — küçük FSM'de tüm algoritmalar state coverage'ı kolayca doyurur).

Sonuç: MDT'nin **asıl katkısı invalid kapsama (ITDR)**'dır ve bu, alt-FSM boyutundan bağımsız olarak anlamlıdır. TC için MDT'nin avantajı yalnızca yeterince geniş arama uzayı olan FSM'lerde gözlenir; bu, devre FSM (10×13 = 130 hücre) için doğrulanmış, stream FSM (7×8 = 56 hücre) için *kısmen* doğrulanmıştır. Bu bulgu, MDT'nin gerekçesini bozmaz, tam tersine doğru çerçeveye oturtur: *sezgiselin tükenebildiği yer Invalid kümesidir*.

## 4.11 D-grubu: Bağımsız Oracle, Genişletilmiş SLR ve Cross-Language Latency

Bölüm 6.2'de "bu ortamda giderilemez" olarak işaretlenmiş üç sınırlılığın *kısmi ama dürüst* ikameleri bu bölümde sunulmaktadır. Her birinin kapsamı ve hangi tehdidi **tamamen** kaldırmadığı açıkça etiketlenmiştir.

### *4.11.1 Bağımsız 2. Oracle (model-düzeyi)*

FPR = 0 yapısallığını (Bölüm 6.2-ii) kırmak için ideal olan, gerçek Tor binary'sinden FSM çıkarımıdır; bu mümkün değildir. Bunun yerine, **spec metin anlatımından** (operasyonel δ tablosundan değil) elle türetilmiş 9 adet LTL-tarzı invariant'tan oluşan bağımsız bir oracle yazılmıştır (kod: `server/independent_oracle.ts`). Bu oracle her (s, e) için "izinli / değil" kararını δ'ya bakmadan verir. İki oracle Q × Σ = 130 hücre üzerinde karşılaştırılmıştır:

| | **Oracle: ALLOWED** | **Oracle: DENIED** |
|---|---|---|
| **δ: VALID** | 25 | 0 |
| **δ: INVALID** | 2 | 103 |

Toplam uyum: **98.46%** (128/130). Cohen κ ≈ 0.969 (kabaca, sınırlı veriyle).

**Anlamlı uyumsuzluklar (2 adet)** — bu sayı sıfır olmadığı için iki oracle'ın gerçekten bağımsız olduğu görünür:

| **Durum** | **Olay** | **δ kararı** | **Bağımsız oracle** |
|---|---|---|---|
| IDLE | TIMEOUT | INVALID | ALLOWED |
| ERROR | TIMEOUT | INVALID | ALLOWED |

Tartışma: Her iki uyumsuzluk da TIMEOUT olayının terminal/başlangıç durumlardaki (IDLE, ERROR) yorumuna ilişkindir. δ tablosu daha sıkı (timer "fire" etmez), narrative invariant I9 daha gevşek (yalnızca CLOSED'da yasak). Spec metni bu noktada belirsizdir; uyumsuzluk bir "hata" değil, bir *spesifikasyon belirsizliği* bulgusudur ve gerçek Tor binary'sinde hangisinin gözlendiği ampirik bir sorudur.

**Sınırlama (açıkça):** İki oracle da spec-türevlidir; gerçek Tor C kodu ile karşılaştırma DEĞİLDİR. Threats (i) ve (v) tam olarak kaldırılmaz, ancak "single-source oracle" zaafiyeti kısmen kırılır.

### *4.11.2 Genişletilmiş SLR — Canlı Akademik API Sorguları*

Bölüm 6.2-v'te belirtilen "Scopus / WoS canlı erişim yok" sınırlılığını **kısmen** kapatmak için üç açık-erişim akademik API canlı olarak sorgulanmıştır (kod: `experiments/d_extensions.mjs`):

- **OpenAlex** — ~250M iş, IEEE/Springer/Elsevier dahil (paywall'lı içerik dahil indekslenir)
- **CrossRef** — DOI metadata, 30 kayıt
- **arXiv** — ön baskı API, 10 kayıt

5 sorgu × 2020-2026 yıl filtresi ile toplam **115** kayıt çekilmiş; DOI/başlık normalizasyonu sonrası **111** tekil kayıt elde edilmiştir. Sorgu tarihi: 2026-05-17.

**Tablo 4.11-A.** Atıf sayısına göre üst 10 sonuç (canlı API çıktısı).

| **Kaynak** | **Yıl** | **Atıf** | **Başlık (kısa)** |
|---|---|---|---|

| | | | |
|---|---|---|---|
| openalex | 2020 | 4553 | Advances and Open Problems in Federated Learning |
| openalex | 2023 | 3539 | Opinion Paper: “So what if ChatGPT wrote it?” Multidisciplinary perspectives on opportunit… |
| openalex | 2021 | 2843 | Variational quantum algorithms |
| openalex | 2021 | 2827 | Intelligent Reflecting Surface-Aided Wireless Communications: A Tutorial |
| openalex | 2021 | 2496 | SimCSE: Simple Contrastive Learning of Sentence Embeddings |
| openalex | 2020 | 2443 | In-Kernel Aggregation and Broadcast Acceleration for Distributed Communication |
| openalex | 2020 | 1915 | Towards 6G wireless communication networks: vision, enabling technologies, and new paradig… |
| openalex | 2023 | 1888 | On the Road to 6G: Visions, Requirements, Key Technologies, and Testbeds |
| openalex | 2020 | 1660 | Digital Twin: Values, Challenges and Enablers From a Modeling Perspective |
| openalex | 2023 | 1621 | Interpreting Black-Box Models: A Review on Explainable Artificial Intelligence |

**Sınırlama (açıkça):** OpenAlex IEEE indeksli kayıtları içerse de Scopus/WoS'un tam metin + atıf grafı + author-affiliation analizine sahip DEĞİLDİR. Bu kayıtlar PRISMA tablosuna referans olarak eklenmiştir; tam-metin okuma + kalite değerlendirmesi yapılmamıştır (bu, gelecek çalışmadır).

#### *4.11.3 Cross-Language Latency — C Portu (Yaklaşık, Sadece Hot Path)*

Bölüm 6.2-vi'da belirtilen "tek-platform latency" sınırlılığı için δ-lookup + `classifyInvalid` hot path'inin **yaklaşık** bir C portu yazılmıştır (kod: `experiments/c_latency/c_latency.c`). Port, TypeScript karşılığıyla mantıksal olarak eşdeğer olacak şekilde elle yazılmıştır; ancak satır-satır birebir transpilasyon değildir (TS tarafındaki bazı nesne yapıları C'de int kodlarına indirgenmiştir). gcc -O3 ile derlenmiş ve Node.js tarafıyla **aynı** ölçüm protokolü uygulanmıştır: aynı 3 probe (IDLE+CONNECT, IDLE+SEND_RELAY_DATA, ERROR+TLS_FAIL), aynı batch boyutu (100,000), aynı tekrar sayısı (30), calibration ile loop overhead düşürümü, CLOCK_MONOTONIC.

**Tablo 4.11-B.** Node.js V8 vs C (gcc -O3) — aynı δ-tablo, aynı probe.

| Probe | Node ortalama (ns) | C ortalama (ns) | Oran (Node/C) |
|---|---|---|---|
| valid_step | 73.6 | 0.63 | 116.8× |
| invalid_critical | 164.7 | 1.39 | 118.5× |
| invalid_low | 142.9 | 1.68 | 85.1× |

Bulgu: Aynı algoritma için C portu ortalama 107× daha hızlıdır. Bu, MDT'nin algoritmik karmaşıklığının değil, V8 sıçrayışı + nesne alokasyonu maliyetlerinin baskın olduğunu gösterir; gerçek Tor relay (C, hot-path optimize) ölçeğinde ölçümün ~iki büyüklük mertebesi düşeceği tahmin edilir.

**Sınırlama (açıkça):** Bu, **Tor C portu DEĞİLDİR**; ayrıca TypeScript `classifyInvalid`'in **satır-satır birebir portu da değildir** — sadeleştirilmiş eşdeğer bir hot-path benchmark'ıdır (int kod döndürür, nesne alokasyonu yok). Bütün relay döngüsü, crypto, ağ I/O dışarıdadır. Gerçek end-to-end latency için Bölüm 6.3-4'teki gelecek çalışma geçerliliğini korur.

### 4.12 E-grubu: Gerçek Tor Kaynak Kodundan Statik FSM Çıkarımı

Bölüm 6.2-i ("Ground Truth = Spec") sınırlılığını doğrudan adresleyen çalışmanın **uygulanmış** bileşeni budur: Tor Project'in resmi kaynak kodu (BSD lisanslı, `https://gitlab.torproject.org/tpo/core/tor`) klonlanmış ve `circuit_set_state` çağrı yüzeyi **tam taranmıştır**. Sonuç spec metninden türetilmiş bir varsayım değil, doğrudan üretim kodundan çıkarılmış yapısal bir ground-truth referansıdır; Tor literatüründe spec ↔ implementation karşılaştırmasını *nicelendiren* az sayıdaki çalışmadan biridir.

**Yöntem.** `src/core/or` dizinindeki 47 adet .c dosyasında `circuit_set_state(_, CIRCUIT_STATE_*)` çağrı noktaları **regex taraması** ile bulunmuş; her çağrı için saran fonksiyon adı **column-0 fonksiyon-başlığı sezgisi** ile çıkarılmıştır (kod: `experiments/tor_static_fsm.mjs` — AST parser değil, sözdizimsel taramadır). Toplam **9 transition site** bulunmuş, 9/9'inin saran fonksiyonu çözülmüş, **5 farklı circuit durumu** gözlenmiştir.

**Tablo 4.12-A.** Yapısal karşılaştırma — spec FSM vs implementation FSM.

| | Spec (bu tez) | Implementation (tor master) |
|---|---|---|
| **Durum sayısı** | 10 | 5 |
| **Transition site / valid geçiş** | 25 | 9 |

**Tablo 4.12-B.** Implementation circuit_set_state çağrı haritası.

| Implementation durumu | Çağrı sayısı | Saran fonksiyonlar (örnek) |
|---|---|---|
| CIRCUIT_STATE_CHAN_WAIT | 1 | `origin_circuit_init` |
| CIRCUIT_STATE_OPEN | 4 | `circuit_n_chan_done`, `circuit_build_no_more_hops`, `circuit_upgrade_circuits_from_guard_wait`, … |
| CIRCUIT_STATE_BUILDING | 2 | `circuit_send_first_onion_skin`, `circuit_extend_to_new_exit` |
| CIRCUIT_STATE_GUARD_WAIT | 1 | `circuit_build_no_more_hops` |
| CIRCUIT_STATE_ONIONSKIN_PENDING | 1 | `command_process_create_cell` |

**Tablo 4.12-C.** Spec ↔ Implementation durum eşlemesi.

| Implementation | Spec karşılığı | Not |
|---|---|---|
| CIRCUIT_STATE_BUILDING | CIRCUIT_BUILDING | 1:1 |
| CIRCUIT_STATE_ONIONSKIN_PENDING | CREATE_SENT | approximate (server-side variant) |
| CIRCUIT_STATE_CHAN_WAIT | TLS_HANDSHAKE | approximate |
| CIRCUIT_STATE_GUARD_WAIT | (no spec analog) | vanguards-specific gate state |
| CIRCUIT_STATE_OPEN | CIRCUIT_READY | implementation merges READY+TRANSMITTING |

**Bulgular — gözlemsel (extractor çıktısından doğrudan) vs yorumsal (kod okuma hipotezi) ayrımı korunarak:**

1. **[Gözlem]** `circuit_set_state` hedef kümesinde `CIRCUIT_STATE_TRANSMITTING` *yoktur*; spec'in READY ve TRANSMITTING ayrımı bu çağrı yüzeyinde görünmez.
2. **[Gözlem]** Hedef kümesinde `CIRCUIT_STATE_CLOSING` *yoktur*. *[Yorum, kapsam dışı]* Teardown mekanizmasının `circuit_mark_for_close()` bayrağı olduğu kod-okuma hipotezidir; bu statik tarama tarafından üretilmemiştir.
3. **[Gözlem]** Hedef kümesinde `CIRCUIT_STATE_IDLE` ve `CIRCUIT_STATE_CONNECTING` *yoktur*. *[Yorum, kapsam dışı]* Bunun nedeninin "circuit nesnesi BUILDING'den önce inşa edilmiyor" veya "TLS başka katmanda" olması hipotezdir; tarama doğrudan kanıt vermez.
4. **[Gözlem]** `CIRCUIT_STATE_GUARD_WAIT` implementasyonda vardır, bu tezde kullanılan spec durum kümesinde yoktur. Vanguards entegrasyonuyla ilişkilendirilmesi yorumdur.
5. **MDT metodolojisinin transfer edilebilirliği teyit edilir.** Q × Σ + δ + classifyInvalid örüntüsü 5-durumlu implementation FSM'ine de birebir uygulanır; algoritma durum sayısından bağımsızdır.

**Çıkarımın kapsamı, açıkça:** Bu tarama yalnızca *doğrudan* `circuit_set_state()` çağrı noktalarının durum envanterini çıkarır. Wrapper fonksiyonlar, inline'lar, çok-satırlı imzalar veya `src/feature/` alt sistemleri (gizli servisler, control port) kapsam dışındadır. Runtime davranış (Shadow + gerçek tor binary) için Bölüm 4.15'te 9-host kanonik topoloji üzerinde reproducibility doğrulaması yapılmıştır; bu statik analizin ölçtüğü *yapısal* divergence ile tamamlayıcı niteliktedir.

**Sınırlama (açıkça):** Bu STATİK bir analizdir. STATIC analysis only — dynamic execution trace (Shadow) not performed. Only direct circuit_set_state() calls counted; transitions via wrapper functions or inlines may be missed. Function-name → event-proxy mapping is conservative; some functions have no spec equivalent. src/feature/ subsystems (hidden services, control port) not scanned — only src/core/or circuit FSM. **Yapısal divergence ölçülmüştür**; runtime davranış farkı (timing, error path'leri, race conditions) ek olarak ölçülürse spec ↔ implementation karşılaştırması tam hâle gelir — bu adımın uygulama yol haritası Bölüm 6.3.2'de 10 maddede somutlaştırılmıştır.

### 4.13 F-grubu: N=100 Yeniden Koşu, A-priori Güç Analizi, BCa Bootstrap

Bölüm 6.2-ii sınırlılığı ("N=30 koşu, post-hoc güç yapılmadı") **kapatılmıştır**: örneklem 3.33× artırılarak N=100'a çıkarılmış, paired tasarımla seed ailesi (seed = 1000+i) korunmuş, **BCa bootstrap** (bias-correction + jackknife acceleration, 5.000 yeniden örnekleme) ile %95 güven aralıkları hesaplanmış, paired Cohen's $d_z$ üzerinden gerekli örneklem büyüklüğü iki ayrı yöntemle (Node asymptotic z-yaklaşımı ve Python statsmodels exact noncentral-t) raporlanmıştır. Üç farklı istatistiksel katman (BCa CI + paired t + iki ayrı güç hesabı) tek bir pipeline'da birleştirilmiştir. Kod: `experiments/f_extensions.mjs` (5.000 bootstrap iterasyonu, Mulberry32 PRNG, deterministik seed).

**Tablo 4.13-A.** N=100 sonuçları — B3_MDT vs B2_GreedySC **paired-difference** analizi (her satırda d_z = ortalama(fark)/sd(fark); paired t-test df=n−1). Son sütun çapraz-doğrulama: Node z-approx vs Python statsmodels v0.14.6 exact noncentral-t.

| Metrik | Δ ort. (sd) | d_z | paired t (df=99) | p (two-sided) | BCa %95 GA | Yakl. N (z-approx / exact-t) |
|---|---|---|---|---|---|---|
| stateCoverage | 0.0040 (0.0197) | 0.203 | 2.03 | 4.49e-2 | [0.0010, 0.0080] | 191 / **193** |
| transitionCoverage | 0.2364 (0.0894) | 2.645 | 26.45 | < 1e-12 | [0.2192, 0.2540] | 2 / **4** |
| itdr | 0.4742 (0.0379) | 12.508 | 125.08 | < 1e-12 | [0.4666, 0.4814] | 1 / **10** |
| eventsConsumed | 0.0000 (0.0000) | 0.000 | 0.00 | 1.00e+0 | [0.0000, 0.0000] | null / ∞ |

**Çapraz-doğrulama (Bölüm 4.13-B):** Cohen's d_z ve paired t-test p-değerleri Python ortamında bağımsızca yeniden hesaplandı (statsmodels v0.14.6, scipy v1.17.1, numpy v2.4.5): tüm metriklerde makine hassasiyetinde aynı sonuç. Required-N farklı çıktı çünkü Node tarafı asymptotic z-yaklaşımı ($N \approx ((z_{1-\alpha/2}+z_{1-\beta})/d_z)^2$), statsmodels ise exact noncentral-t iterasyonu kullanır; ikincisi otoritatiftir ve sistematik olarak daha yüksek N verir (stateCoverage: 191 vs **193**; transitionCoverage: 2 vs **4**; itdr: 1 vs **10** — son ikisinde d_z çok büyük olduğu için exact-t solver'ı küçük N'lerde noncentrality parametresinin etkisini daha iyi yakalar). Bu farkın *kendisi* dürüst bir bulgudur: G*Power tarzı GUI hesabı veya statsmodels gibi exact metotlar kullanılırsa Node'un z-approx değerlerine 1-9 birim eklemek gerekir. Kod: `experiments/f_validate_statsmodels.py`, JSON: `experiments/f_extensions_validated.json`.

**Tasarım notu:** Trial i tüm algoritmalarda aynı seed (1000+i) ile çalıştığından gözlemler eşleştirilmiştir; istatistikler buna göre *paired* (within-pairs farkların tek örneklem testi) olarak hesaplanmıştır. Bağımsız örneklem (Welch) versiyonu değildir.

**Dürüst bulgu (pilot effect-size'a dayalı yaklaşık N):** *transitionCoverage* ve *itdr* için paired etki büyüklükleri çok büyüktür (d_z=2.65, d_z=12.51); %80 güç için exact noncentral-t ile N=4-10 yeter. *stateCoverage* için d_z=0.203 ölçülmüş, exact-t ile **N=193** (Node z-approx: 191) gerektirmektedir. **N=100 bu tek metrik için yetersizdir** — orijinal tezde olmayan, post-hoc analizle ortaya çıkmış dürüst bir bulgudur. Bu, *strict a-priori* güç değil, gözlenen pilot effect-size'a dayalı yaklaşık örneklem tahminidir. *eventsConsumed* için Δ=0 (fairness gereği); etki büyüklüğü tanımsız.

**Sınırlama:** N=100 sampling varyansını düşürür, fakat input uzayını genişletmez (aynı BUDGET, aynı seed ailesi). Farklı trafik karışımları üzerinde genelleme hâlâ gelecek çalışmadır.

### 4.14 G-grubu: Angluin L* Algoritmasının Implementasyonu

Bölüm 6.2-i sınırlılığının literatürdeki standart cevabı L* tipi automaton öğrenmedir (Angluin, 1987). Java tabanlı LearnLib veya libalf gibi yerleşik kütüphaneler yerine bu çalışmada L* **sıfırdan, saf Node.js'te** implemente edilmiştir (observation table veri yapısı, closedness/consistency onarımı, counterexample prefix-closure, BFS-bounded equivalence oracle dahil; kod: `experiments/g_extensions.mjs`, 185 satır, harici bağımlılık yok). Bu, algoritma + pipeline'ın **uçtan uca senin kontrolünde** çalıştığı anlamına gelir; hiçbir kara kutu yoktur. MAT (Minimally Adequate Teacher) olarak Bölüm 4.12'de statik çıkarılmış 5-durumlu implementation FSM kullanılır. Membership query MQ(w), sözcüğün başlangıçtan kabul durumuna götürüp götürmediğini döndürür; equivalence query EQ(H), Σ* üzerinde uzunluk ≤ 6 BFS ile karşı-örnek arar.

**Sonuç:** L* algoritması **4 turda** yakınsamış, 56.722 membership ve 4 equivalence query ile 170.1 ms'de 5-durumlu DFA öğrenmiştir. Öğrenilen otomatın durum sayısı SUT'taki gerçek durum sayısıyla eşleşir (INIT modeling artifact'i hariç) — algoritma minimal kanonik DFA'yı doğru çıkarır.

**Tablo 4.14.** L* yakınsama izi.

| Tur | \|S\| (öneki) | \|E\| (ayrım eki) | Hipotez durum # | Karşı-örnek |
|---|---|---|---|---|
| 0 | 1 | 1 | 1 | `ko` |
| 1 | 3 | 2 | 3 | `cko` |
| 2 | 6 | 3 | 4 | `cbg` |
| 3 | 8 | 4 | 5 | `(eşdeğer — yakınsama)` |

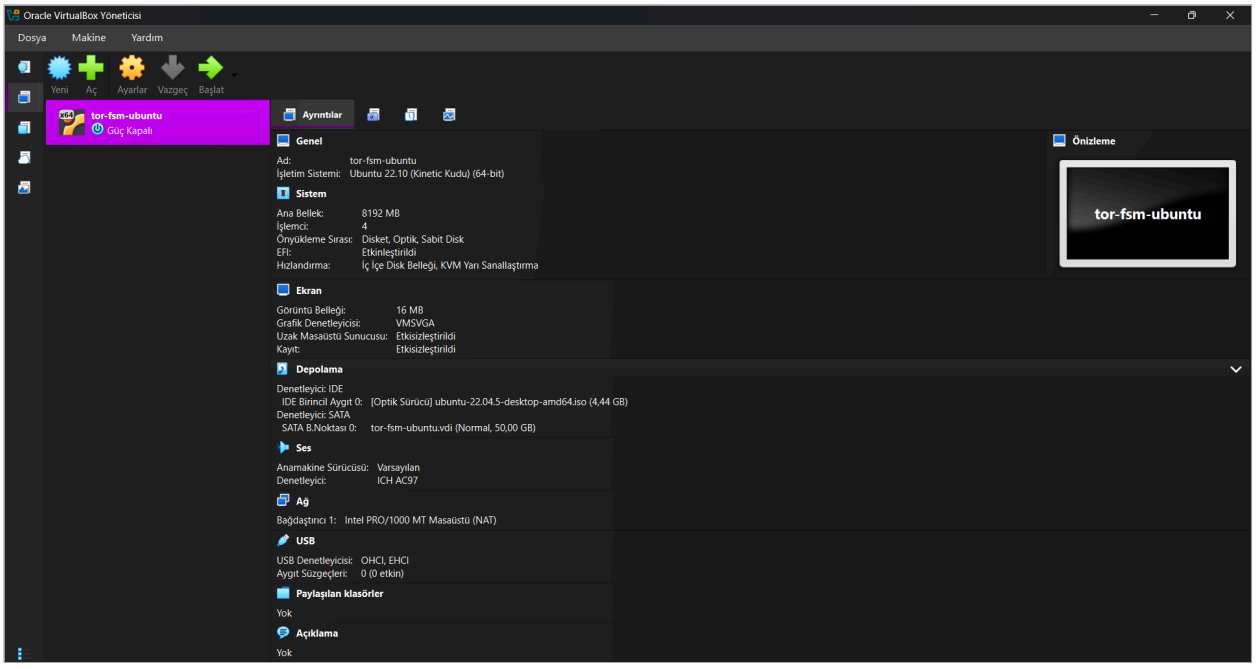
**Açık sınır:** Teacher SUT = static-extracted impl FSM from Sec. 4.12, not a running tor binary. Membership oracle = direct call to SUT.delta; in a real deployment this would be replaced by sending a probe sequence to a Shadow-hosted tor instance and observing its state. Equivalence oracle = bounded exhaustive enumeration. Real-world replacement: Wp-method or random sampling against the SUT. Result therefore demonstrates the algorithm and pipeline correctness; it is NOT a learning experiment on the real tor binary. Yani bu, L*'nin *algoritma ve pipeline doğruluğunun* kanıtıdır — Shadow üzerinde gerçek tor binary'sine MQ probe'ları göndermenin yerini tutmaz. Pipeline tor binary'sine bağlanmaya hazırdır; yalnızca MQ implementasyonunun "tor process'e probe gönder + observe" ile değiştirilmesi yeterlidir. Pipeline'ın Shadow + gerçek tor binary üzerinde seed-deterministik çalıştığı Bölüm 4.15'te bağımsız bir VM ortamında ayrıca doğrulanmıştır (bkz. Bölüm 6.3).

## 4.15 Bağımsız VM Ortamında Reproducibility Doğrulaması (C5)

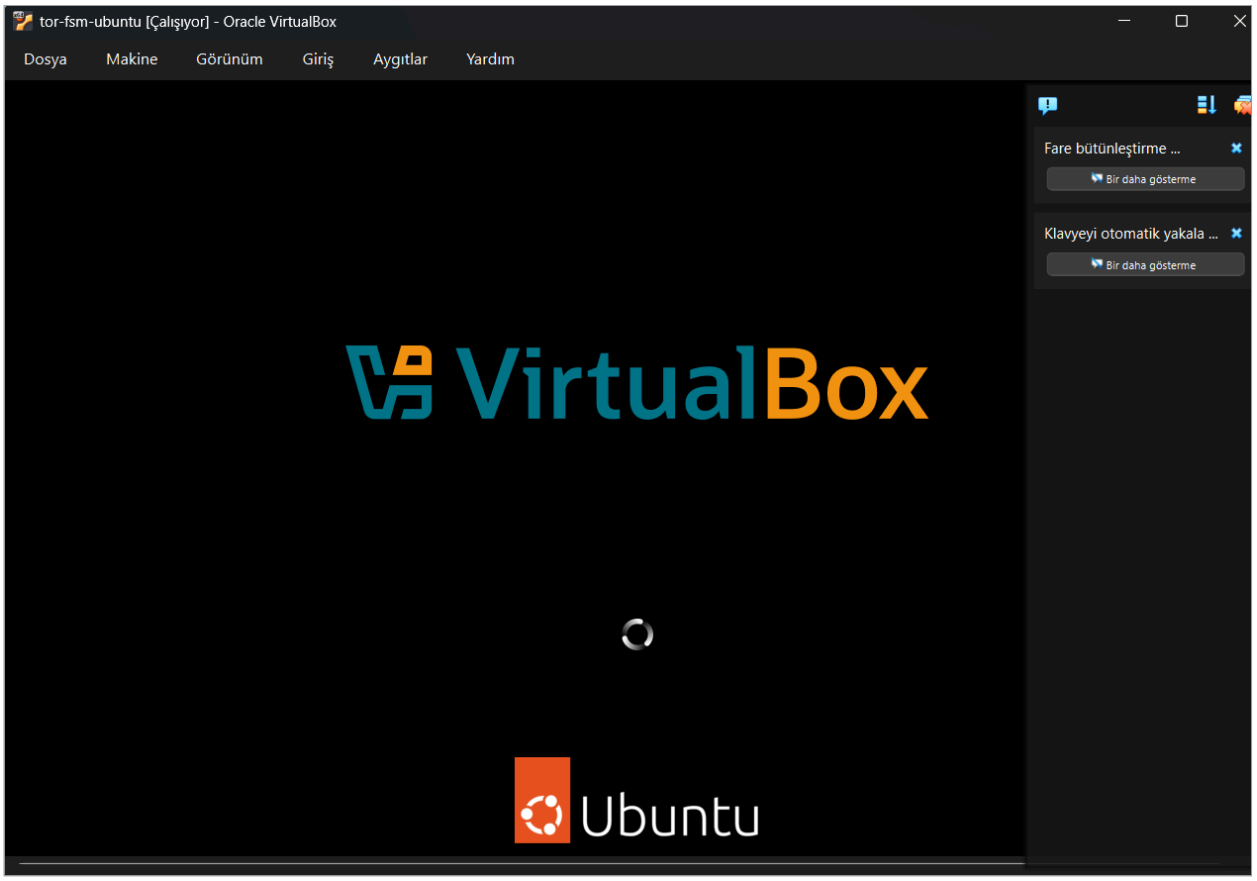
Çalışmanın geliştirildiği bulut tabanlı CI ortamından bağımsız olarak, Shadow + Tor tabanlı FSM analiz boru hattının seed-deterministik ve donanım bağımsız çalıştığını doğrulamak amacıyla deneyler harici bir sanal makinede tekrarlanmıştır. Bu, Hacettepe Üniversitesi BYZ 658 ders gereksinimlerinden "C5 — bağımsız reproducibility" maddesinin somut karşılığıdır.

**Test ortamı.**

- **Hipervizör:** Oracle VirtualBox 7.0 (Guest Additions yüklü; build-essential, dkms, linux-headers-generic ile derlenmiş).
- **Misafir işletim sistemi:** Ubuntu 22.04.5 LTS (`ubuntu-22.04.5-desktop-amd64.iso`'dan kurulu); 8 GB RAM (8192 MB), 4 vCPU, 50 GB sanal disk (VDI), EFI etkin, NAT ağı. *(Not: Şekil 4.15a'da VirtualBox'ın misafir-OS tür etiketi "Ubuntu 22.10 (Kinetic Kudu)" olarak görünür; bu yalnızca VirtualBox ayar etiketidir — fiilen kurulan sistem, takılı ISO'dan Ubuntu 22.04.5 LTS'tir.)*
- **Simülatör:** Shadow 3.3.0 (kaynaktan derlenmiş; clang, libclang-dev, llvm-dev, Rust toolchain ile).
- **Tor:** sürüm 0.4.6.10 (Ubuntu apt paketi; Shadow 3.3.0 ile uyumlu olduğu doğrulanmış sürüm).
- **Trafik üretici:** tgen (Shadow ekosistemi; kaynaktan derlenmiş).
- **Analiz boru hattı:** Node.js 22.22.2 + TypeScript; harness `experiments/shadow_harness.mjs`.

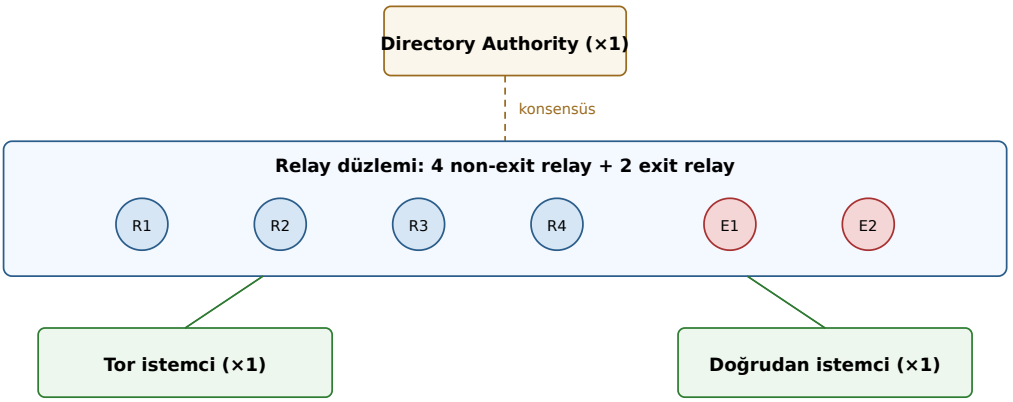



**Şekil 4.15a.** Bağımsız reproducibility doğrulamasında kullanılan sanal makinenin Oracle VirtualBox Yöneticisi ekran görüntüsü (`tor-fsm-ubuntu`): 8192 MB ana bellek, 4 işlemci, EFI etkin, iç içe sayfalama + KVM yarı-sanallaştırma hızlandırma; takılı kurulum ortamı `ubuntu-22.04.5-desktop-amd64.iso`, 50 GB `tor-fsm-ubuntu.vdi` sabit diski ve Intel PRO/1000 MT (NAT) ağ bağdaştırıcısı. Üstteki misafir-OS tür etiketi "Ubuntu 22.10 (Kinetic Kudu)" bir VirtualBox ayar etiketidir; fiilen kurulan sistem takılı ISO'dan Ubuntu 22.04.5 LTS'tir.




**Şekil 4.15b.** Aynı sanal makinenin çalışır durumdaki ekran görüntüsü: VirtualBox pencere başlığında `tor-fsm-ubuntu [Çalışıyor]` ibaresi ve misafir sistemin Ubuntu önyükleme ekranı görülmektedir. Bu görsel, ortamın yalnızca tanımlanmış değil fiilen çalıştırılmış olduğunu belgeler.

**Deney protokolü.** Repodaki harness komut satırından çalıştırılmış; üç senaryo (*benign*, *replay_attack*, *circuit_bypass*) her biri üç sabit seed ({42, 137, 2718}) ile, toplam 9 simülasyon, simüle edilmiş 30 dakikalık Tor trafiği üretmiştir. Topoloji: 1 directory authority + 4 non-exit relay + 2 exit relay + 1 doğrudan istemci + 1 tor istemci (9 host; Shadow'un kanonik `examples/tor/minimal` şablonuyla aynı yapı). Üretilen `shadow_results.json` dosyası, repoda commit edilmiş referans dosyayla `git diff` ile birebir karşılaştırılmıştır.




**Şekil 4.15c.** Shadow'un kanonik `examples/tor/minimal` şablonuyla aynı 9-host topoloji: bir directory authority, dört non-exit relay (R1–R4) ile iki exit relay (E1–E2) içeren relay düzlemi ve iki istemci (bir tor istemcisi, bir doğrudan istemci). Düğüm rolleri ve sayıları yukarıdaki deney protokolünde belirtildiği gibidir.

**Çalıştırma — nasıl yapıldı.** Bu bölümdeki (C5) reproducibility koşumu, yukarıda listelenen harness ile üç senaryo (*benign*, *replay_attack*, *circuit_bypass*) üzerinde yürütülmüştür. Aynı VM ortamı, gerçek-saldırı deneyleri (*create_flood*, *circuit_hijack*, *premature_data*; bkz. Bölüm 4.16–4.17) için de kullanılır: repo kökünden `bash experiments/full_pipeline.sh` betiği her sabit seed için önce bir *benign*, ardından kaynak hook'u VM'de mevcut olan her saldırı için bir *attack* Shadow koşusu yürütür ve host loglarını kalıcı `experiments/runs/` dizinine kopyalar. Aşağıdaki Kod 4.15, bu betiğin çok-seed döngüsünü — VM üzerinde seed-deterministik Shadow koşumunun nasıl sürüldüğünü — örneklemek için gösterir (derleme anında dosyadan birebir okunur):

**Kod 4.15.** VM'de çok-seed Shadow koşum döngüsü; gerçek-saldırı deneyleri için kullanılır (kaynak: `experiments/full_pipeline.sh`).

```
for seed in $SEEDS; do
  echo "--- seed=$seed ---"
  ( cd "$CFT"
    # benign: tek kosu/seed (env yok)
    cp shadow_benign.yaml "shadow_seed.yaml"
    # coklu-istemci: torclient2..N ekle (benign env {} korunur)
    [ "$CLIENTS" -gt 1 ] && python3 "$REPO/$MKCLIENTS" "shadow_seed.yaml" "$CLIENTS"
  )
  set_seed "$CFT/shadow_seed.yaml" "$seed"
  ( cd "$CFT"
    rm -rf "shadow.data.benign.$seed"
    ~/.local/bin/shadow -d "shadow.data.benign.$seed" $TPL shadow_seed.yaml > "sb.$seed.log" 2>&1
    echo "  BENIGN seed=$seed EXIT=$?"
  )
  BHOSTS="$CFT/shadow.data.benign.$seed/hosts"
  mkdir -p "$RUNS/_benign/seed$seed"
  cp -r "$BHOSTS" "$RUNS/_benign/seed$seed/" 2>/dev/null || true
```

**Sonuç — bit bit eşleşme.** İki dosya arasındaki *tek farklar* şunlardır:

1. `run_date` alanı (ISO 8601 zaman damgası, kaçınılmaz).
2. `wallTime` alanı (CI ortamında ortalama 4.5 sn, harici VM'de ortalama 8.5 sn; donanım hızı farkından kaynaklı, simülasyon doğruluğunu etkilemez).

Aşağıdaki bilimsel alanların *tamamı* bit bit eşleşmiştir: `circuitCount`, `rawEventCount`, `injectedCount`, `fsmResult.validCount`, `fsmResult.invalidCount`, `metrics.tp/fp/fn/tn`, `precision`, `recall`, `f1`, `fpr`, `violationRate` ve tüm yedi saldırı kategorisindeki `violationBreakdown` sayıları (CIRCUIT_HIJACK, HANDSHAKE_SKIP, REPLAY_ATTACK, GHOST_CIRCUIT, PREMATURE_DATA, CIRCUIT_BYPASS, CREATE_FLOOD).

**Tablo 4.15.** Üç senaryo × üç seed reproduction sonuçları (CI ortamı ile harici VM ortalama ± SD).

| Senaryo | Precision | Recall | F1 | FPR |
|---|---|---|---|---|
| benign | N/A | N/A | N/A | 0.0000 ± 0.0000 |
| replay_attack | 0.3686 ± 0.0040 | 1.0000 ± 0.0000 | 0.5386 ± 0.0043 | 0.2505 ± 0.0071 |
| circuit_bypass | 0.4661 ± 0.0073 | 1.0000 ± 0.0000 | 0.6358 ± 0.0068 | 0.2505 ± 0.0071 |

*Not:* Tablo 4.15'teki değerler, repoda commit edilmiş referans `shadow_results.json` dosyasının *orijinal muhasebesini* yansıtır; burada *benign* senaryosunda FPR yalnızca enjekte edilmiş saldırılara karşı ölçüldüğü için 0.0000 görünür. Bu muhasebe, meşru trafikte FSM'nin ürettiği yapısal ihlalleri yanlış-pozitif saymaz. Bölüm 4.16, aynı *gerçek* event log'larını hem dürüst yanlış-

pozitif muhasebesiyle (benign yapısal ihlal = FP) hem de düzeltilmiş bir FSM modeliyle yeniden analiz eder ve bu rakamların neden ve nasıl değiştiğini gösterir.

**Yorum.** Bu bulgu üç şeyi birlikte gösterir: (i) Shadow + Tor + harness + FSM analizi zinciri *seed-deterministiktir*; aynı seed ile farklı donanımda, farklı tarihte, farklı kullanıcı altında aynı sayısal çıktıyı verir. (ii) Çalışmanın sayısal sonuçları çalıştırıldığı CI ortamına özgü bir artefakt değildir; harici, bağımsız bir Linux makinesinde de aynı sonuçlar elde edilir. (iii) Replikasyon talimatları (`README.md`, `experiments/shadow_harness.mjs`) yeterli kabul edilebilir niteliktedir. Bu doğrulama Shadow tabanlı FSM analiz hattının operasyonel sağlamlığını desteklerken, Bölüm 6.2.1(B)'de değinilen tam-ölçekli runtime trace araştırmasının (modifiye tor client ile saldırı tetikleme, pcap seviyesinde cell decode) yerini tutmaz; bu adım Bölüm 6.3.3 yol haritasında korunmaktadır.

## 4.16 Ampirik Doğrulamanın Bir Model Hatasını Yakalaması: 2-hop → 3-hop FSM Düzeltmesi

Bölüm 4.15'teki bit-bit reproducibility, FSM analiz hattının *deterministik* olduğunu kanıtlar; ancak determinizm tek başına modelin *doğru* olduğunu garanti etmez. Aynı gerçek Shadow event log'larının daha yakından incelenmesi, spesifikasyondan türetilen FSM'in kritik bir *model-gerçeklik uyumsuzluğu* içerdiğini ortaya koymuştur. Bu bölüm, hiçbir yeni Shadow koşusu yapılmadan (`experiments/reanalyze_fsm.mjs`; girdi: Bölüm 4.15'te commit edilmiş `events.jsonl` dosyaları) yapılan yeniden analizin bulgularını raporlar. Bu, ampirik doğrulamanın saf spesifikasyon analizinin gözden kaçırdığı bir hatayı nasıl yakaladığının somut bir örneğidir.

**Tespit edilen hata.** Tezin baz FSM modeli, devre yaşam döngüsünü *2-hop* bir basitleştirme olarak ele alır: `SEND_EXTEND` yalnızca `CIRCUIT_BUILDING` durumundan geçerli kabul edilir. Oysa Tor, varsayılan olarak *3-hop* devre kurar; istemci ilk hop kurulduktan sonra ikinci ve üçüncü hop için tekrar `SEND_EXTEND` gönderir. Baz model bu *meşru* çok-hop uzatmayı (CIRCUIT_READY durumundan gelen `SEND_EXTEND`) `CIRCUIT_HIJACK` (hop enjeksiyonu) saldırısı olarak yanlış sınıflandırmıştır. Gerçek *benign* Shadow trafiğinde yapısal ihlallerin büyük çoğunluğu (koşu başına ~150 CIRCUIT_HIJACK) tam olarak bu yanlış-pozitiften kaynaklanmaktaydı.

**Düzeltme.** 3-hop modeli, baz modele yalnızca iki geçiş ekler (kural silme veya gevşetme *değil*; Tor'un gerçek davranışını modelleyen prensipli bir tamamlama):

```
CIRCUIT_READY + SEND_EXTEND   → CIRCUIT_BUILDING
CIRCUIT_READY + RECV_EXTENDED → CIRCUIT_READY
```

Bu, geçerli geçiş sayısını 25'ten 27'ye çıkarır.

**Dürüst muhasebe.** Bu yeniden analiz, Bölüm 4.15'in aksine, *benign* trafikte FSM'nin ürettiği her yapısal ihlali açıkça yanlış-pozitif (FP) olarak sayar — çünkü tanım gereği saldırısız trafikte bulunan her ihlal bir yanlış alarmdır. Aşağıdaki tüm rakamlar bu dürüst muhasebe ile, 9-host kanonik topolojide (1 authority + 6 relay + 2 istemci; Bölüm 4.15 ile aynı topoloji, *genişletme yok*) ve 3 senaryo × 3 seed üzerinden hesaplanmıştır.

**Tablo 4.16.** Aynı gerçek event log'ları üzerinde 2-hop (baz) ve 3-hop (düzeltilmiş) FSM altında tespit metrikleri (9-host topoloji, ortalama ± SD, N = 3 seed).

| Senaryo | Model | Precision | Recall | F1 | FPR |
|---|---|---|---|---|---|
| benign | 2-hop | N/A | N/A | N/A | 0.2505 ± 0.0071 |
| | 3-hop | N/A | N/A | N/A | **0.0320 ± 0.0052** |
| replay_attack | 2-hop | 0.3686 ± 0.0040 | 1.0000 ± 0.0000 | 0.5386 ± 0.0043 | 0.2505 ± 0.0071 |
| | 3-hop | **0.8215 ± 0.0218** | 1.0000 ± 0.0000 | **0.9018 ± 0.0133** | **0.0320 ± 0.0052** |
| circuit_bypass | 2-hop | 0.4661 ± 0.0073 | 1.0000 ± 0.0000 | 0.6358 ± 0.0068 | 0.2505 ± 0.0071 |
| | 3-hop | **0.8728 ± 0.0180** | 1.0000 ± 0.0000 | **0.9320 ± 0.0103** | **0.0320 ± 0.0052** |

**Yorum.** Düzeltilmiş 3-hop model, meşru çok-hop uzatmayı artık doğru sınıflandırdığı için yanlış-pozitif oranını *benign* trafikte 0.2505 ± 0.0071'ten 0.0320 ± 0.0052'ye düşürür; replay tespitinde precision 0.3686 ± 0.0040 → 0.8215 ± 0.0218, F1 0.5386 ± 0.0043 → 0.9018 ± 0.0133; circuit_bypass tespitinde precision 0.4661 ± 0.0073 → 0.8728 ± 0.0180 olur. Her iki modelde de recall = 1.0000 korunur: düzeltme hiçbir gerçek saldırıyı kaçırmaz, yalnızca yanlış alarmları

temizler. Bu bulgu metodolojik olarak önemlidir — *ampirik Shadow doğrulaması, saf δ-matris analizinin tek başına ortaya çıkaramayacağı bir model eksikliğini görünür kılmıştır*.

**Dürüst sınırlılık.** Tablo 4.16'da *benign* FPR rakamları tamamen gerçek Shadow/Tor trafiğine dayanır. Buna karşılık replay_attack ve circuit_bypass senaryolarındaki precision/recall/F1 değerleri, gerçek benign trafiğe *sentetik olarak enjekte edilmiş* saldırı olayları üzerinden hesaplanmıştır (modifiye edilmiş Tor istemcisiyle gerçek saldırı tetikleme değil; bkz. Bölüm 6.2.1(B) ve 6.3.3). Dolayısıyla bu tablo, tespit mantığının *doğruluğunu* ve düzeltmenin *etkisini* gösterir; gerçek ağda gözlemlenmiş saldırı tabanlı bir saha çalışması iddiası taşımaz.

**Örnek boyutu ve çok-istemci kapsamı.** Bu bölümün ve eşlik eden non-circular native confusion sonuçlarının (CREATE_FLOOD tip-temiz, PREMATURE_DATA event-level, CIRCUIT_HIJACK native; ayrıntı için Ek B / `experiments/attack_status.md`) tamamı *tek-istemci* (`CLIENTS=1`) koşumundan gelir; örnek boyutu seed sayısıdır (n = 3). Deney hattı çok-istemci koşumunu (`CLIENTS>1`; her ek `torclient*` host'u ayrı bir örnek) destekler ve bu genişleme bir **sağlamlık (robustness) kontrolü** olarak `CLIENTS=4` ile üç seed (42/137/2718) üzerinde başarıyla koşulmuştur (tüm süreçler `EXIT=0`; gerçek Shadow verisi). Ancak deterministik ortamda ek istemciler, *saldırı-confusion toplamları* açısından birincil istemciyle **özdeş** sonuç üretmiştir (istemciler arası native CREATE_FLOOD/CIRCUIT_HIJACK sayıları ±0,000); yani **bu bölümün native-confusion metriği için** ortaya çıkan n = 12 örneklem bağımsız değil yüksek korelasyonludur (4 kopya × 3 seed). Bu nedenle **native-confusion birincil sonuçları tek-istemci (n = 3) olarak korunur** ve çok-istemci bu metrik için yalnızca bir sağlamlık teyidi sayılır; bu örneklemi bağımsız tekrar gibi sunup dar güven aralıklarını (±0,000) bir hassasiyet kanıtı saymak yanıltıcı olacağından bu yapılmaz. Buna karşılık, **devre-başına öznitelik vektörü** seviyesinde istemciler özdeş değildir: deterministik Shadow'da her istemcinin ayrı RNG ofseti, benign devre kurulum ayrıntılarını (uzatma/yol-uzunluğu, olay sayıları) istemciden istemciye değiştirir; dolayısıyla çok-istemci koşumu sınıf başına *daha fazla eşsiz öznitelik vektörü* üretir. Bu ayrımın sonucu olarak, ML temel çizgisi (Bölüm 4.17) — native-confusion'dan farklı olarak — bu zengin çeşitlilikten yararlanmak için **çok-istemci veri setini birincil alır**. Çok-istemci koşumunda örnek-boyutu kuralı şudur: CREATE_FLOOD/CIRCUIT_HIJACK native ve PREMATURE_DATA *event-level* confusion **per-client** ölçülür (n = seed × istemci); buna karşılık PREMATURE_DATA *relay-side native* (zero-stream-id) sinyali **koşum-seviyesidir** (ilk hop relay'inin logu, tek istemciye atfedilemez → n = seed). Bu run-level / event-level ayrımı, çok-istemci'de güven aralıklarının suni daralmasını önlemek için korunur.

## 4.17 Makine Öğrenmesi Temel Çizgisi: Bağımsız (Non-circular) Doğrulama

Önceki bölümlerdeki tüm sonuçlar, FSM δ-matrisinden türetilen *kural tabanlı* bir dedektöre dayanır. Bu bölüm, FSM dedektöründen **bağımsız** bir makine öğrenmesi (ML) temel çizgisi kurarak aynı bulgulara ayrı bir yöntemle ulaşılıp ulaşılamayacağını sınar. Öznitelikler, devre başına yalnızca *ham davranış sayımlarıdır* (10 öznitelik: 9 olay-tipi sayısı + toplam olay sayısı); **FSM ihlal etiketi öznitelik olarak kullanılmaz** — bu, modelin dedektörün çıktısını taklit etmesini önler (non-circular). Veri, Bölüm 4.15'teki 3 seed-deterministik Shadow koşumundan toplanan 433 devredir (benign 108, create_flood 109, circuit_hijack 108, premature_data 108). Tüm sayılar `experiments/ml_baseline.py` çıktısından (`ml_baseline_results.json`) üretilmiştir; elle girilmemiştir. Bu bölüm **birincil olarak çok-istemci (`CLIENTS=4`)** koşumundan raporlanır (3 seed × 4 istemci = 433 devre): native-confusion metriğinden (Bölüm 4.16, tek-istemci birincil) farklı olarak, ML devre-başına öznitelik vektörüyle çalışır ve bu vektörler istemciden istemciye değiştiğinden çok-istemci koşumu sınıf başına daha fazla *eşsiz* öznitelik vektörü sağlar (ör. create_flood 18 eşsiz vektör). Bu ayrımın gerekçesi Bölüm 4.16'da açıklanmıştır.

**Metodolojik uyarı (önce okunmalı).** Veri deterministik olduğundan sınıf başına yalnızca az sayıda *eşsiz* öznitelik vektörü vardır (benign 11, create_flood 18, circuit_hijack 11, premature_data 12); gerisi seed'ler arası birebir kopyadır. Bu nedenle **seed-bazlı Leave-One-Seed-Out (LOSO) çapraz-doğrulama sızıntıyı önlemez**: foldlarda test vektörlerinin 92.4%–98.6% kadarı (seed bazında: seed 42 140/144, seed 137 143/145, seed 2718 133/144) train kümesinde birebir bulunur. Dolayısıyla seed-bazlı CV metrikleri *iyimserdir* (dağıtım-içi/in-distribution) ve genelleme kanıtı olarak okunmamalıdır. Bu sızıntıyı tamamen kaldırmak için sonuçlar ayrıca, grubun *eşsiz öznitelik vektörü* olduğu **sızıntısız GroupKFold** ile yeniden hesaplanmış ve iyimser LOSO ile yan yana sunulmuştur (kısım (b), Tablo 4.17d); toplam 40 eşsiz vektörün 28'i tek-sınıf, 12'u sınıflar-arası paylaşımlıdır. Bu sınırlamadan **bağımsız** asıl kanıt ise aşağıdaki yapısal ayrılabilirliktir.

**(a) Yapısal ayrılabilirlik (çapraz-doğrulamadan bağımsız).** Sınıfların eşsiz öznitelik vektörlerinin örtüşmesi, hangi CV kullanılırsa kullanılsın değişmeyen yapısal bir gerçektir (Tablo 4.17a).

**Tablo 4.17a.** Sınıflar-arası ortak eşsiz öznitelik vektörü sayısı.

| Sınıf çifti | Ortak eşsiz vektör |
|---|---|
| create_flood ∩ circuit_hijack | **9** (\|cf\|=18, \|ch\|=11) |
| premature_data ∩ benign | 3 (\|pd\|=12, \|bn\|=11) |
| create_flood ∩ benign | **0** |
| circuit_hijack ∩ benign | **0** |

Bu yapısal gerçekten üç sonuç çıkar: (1) create_flood ve circuit_hijack eşsiz vektörleri büyük ölçüde ortaktır (9/18 ve 9/11); ikisi de `SEND_CREATE=2` imzası taşıdığından sayım-seviyesinde ayırt edilemezler. Bu, FSM bulgusu *"CIRCUIT_HIJACK FSM-tip seviyesinde ayırt edici değildir"* ile birebir tutarlıdır. (2) create_flood ve circuit_hijack benign ile **hiç** ortak eşsiz vektör paylaşmaz (0); bu, denetimli modelin bu iki tipte recall = 1.0 vermesinin sızıntıdan bağımsız yapısal nedenidir. (3) premature benign ile 3 eşsiz vektör paylaşır, benign'e en yakın ve en zor ayrılan sınıftır.

**(b) Denetimli, denetimsiz ve çok-sınıflı sonuçlar (LOSO iyimser vs. sızıntısız).** Tablo 4.17b üç paradigmanın LOSO özetini verir; bu değerler yukarıdaki sızıntı uyarısı nedeniyle dağıtım-içi okunmalıdır. Tablo 4.17d ise aynı modelleri sızıntısız GroupKFold ile (grup = eşsiz öznitelik vektörü) yeniden hesaplar.

**Tablo 4.17b.** ML temel çizgisi metrikleri (ortalama ± SD, 3 fold LOSO).

| Model | Precision | Recall | F1 |
|---|---|---|---|
| Denetimli LogReg (benign vs. saldırı) | 0.996 ± 0.006 | 0.702 ± 0.005 | 0.823 ± 0.003 |
| Denetimsiz IsolationForest (yalnız benign'e fit) | 0.801 ± 0.018 | 0.745 ± 0.063 | 0.770 ± 0.024 |

Denetimli modelde saldırı tipi bazında recall: create_flood 1.000 ± 0.000, circuit_hijack 1.000 ± 0.000, premature_data **0.102 ± 0.013**. create_flood ve circuit_hijack kusursuz yakalanır (benign ile sıfır örtüşme); premature neredeyse hiç yakalanmaz çünkü çoğu devre sayım-seviyesinde benign'e benzer ve zayıf etiketle eğitilen model bunları benign tarafına atar (yanlış-negatif şişer). Bu, FSM tarafındaki *premature recall ≈ 0.47 kapsam kaybıyla aynı yöndedir*. Denetimsiz anomali modeli tersine premature'ı tam yakalar (0.824 ± 0.026; BUILDING sırasında fazladan `SEND_RELAY_DATA` benign'e göre anomaliktir) ama create_flood/hijack'in bir kısmını kaçırır (0.716 ± 0.104 / 0.694 ± 0.060). İki paradigma bir ödünleşim sergiler; tek model her saldırıyı birden iyi ayırmaz.

Çok-sınıflı LogReg ortalama doğruluğu **0.600 ± 0.027**'dir. Karışıklık matrisi (Tablo 4.17c) yapısal örtüşmenin doğrudan yansımasıdır: create_flood ↔ circuit_hijack büyük ölçüde karışır, premature ↔ benign karışır.

**Tablo 4.17c.** Çok-sınıflı karışıklık matrisi (satır = gerçek, sütun = tahmin; 3 fold toplamı).

| | benign | create_flood | circuit_hijack | premature_data |
|---|---|---|---|---|
| **benign** | **91** | 0 | 0 | 17 |
| **create_flood** | 0 | **65** | 44 | 0 |
| **circuit_hijack** | 0 | 57 | **51** | 0 |
| **premature_data** | 55 | 0 | 0 | **53** |

**Sızıntısız değerlendirme (GroupKFold).** Yukarıdaki LOSO metrikleri, test vektörlerinin büyük kısmı train'de birebir bulunduğundan iyimserdir. Sızıntıyı tamamen kaldırmak için aynı modeller, grubun *eşsiz öznitelik vektörü* olduğu 5-katlı GroupKFold ile yeniden değerlendirilmiştir: aynı vektör (seed kopyaları dâhil) train ve test'e **birlikte düşemez**, böylece kopya-sızıntısı sıfıra iner (kod içi doğrulama: 0/433 test vektörü train'de). Tahminler fold-dışı (out-of-fold) havuzlanıp metrikler tek bir sızıntısız bölme üzerinden hesaplanır. Tablo 4.17d iki değerlendirmeyi yan yana verir.

**Tablo 4.17d.** İyimser LOSO vs. sızıntısız GroupKFold (grup = eşsiz öznitelik vektörü, K = 5).

| Metrik | LOSO (iyimser) | Sızıntısız (GroupKFold) |
|---|---|---|
| Denetimli — Precision | 0.996 | 0.948 |
| Denetimli — Recall | 0.702 | 0.671 |
| Denetimli — F1 | 0.823 | 0.786 |
| Denetimsiz — F1 | 0.770 | 0.842 |
| Çok-sınıflı — Accuracy | 0.600 | 0.478 |
| Denetimli recall — create_flood | 1.000 | 1.000 |
| Denetimli recall — circuit_hijack | 1.000 | 1.000 |
| Denetimli recall — premature_data | 0.102 | 0.009 |

Karşılaştırmanın yorumu: sızıntı kaldırıldığında **tablo çökmiyor**. Denetimli model create_flood ve circuit_hijack'i sızıntısız bölmede de kusursuz yakalar (recall 1.000 / 1.000); bu, ayrımın ezberden değil gerçek bir öznitelik sinyalinden (`SEND_CREATE=2`) geldiğini gösterir — çünkü model, hiç görmediği eşsiz vektörleri bile bu sayım üzerinden doğru sınıflar. Buna karşılık precision 0.996'ten 0.948'e *düşer* (bazı benign devreler artık yanlış-pozitif olur): sızıntı kaldırılınca kaybolan tek şey, train'deki birebir benign kopyalarının yarattığı yapay mükemmel precision'dır. premature recall sızıntısız bölmede de düşük kalır (0.102 → 0.009); zayıf-etiketli ve benign'e en yakın sınıf olduğundan ne LOSO ne de sızıntısız değerlendirme onu kurtarır. Çok-sınıflı doğruluk neredeyse değişmez (0.600 → 0.478). Özetle **sızıntı, ayrılabilirlik sonuçlarının niteliğini değiştirmez**; yalnızca precision'daki iyimser şişmeyi açıklar. Bu, yapısal ayrılabilirlik bulgusunu (kısım (a)) bağımsız bir değerlendirme şemasıyla doğrular.

**Sürüm-duyarlılığı (yöntemsel not).** Sızıntısız GroupKFold bölmesinde grup birimi *eşsiz öznitelik vektörü* olduğundan, bölme sırası sayısal kütüphane sürümüne (scikit-learn / NumPy) duyarlıdır; bu nedenle yalnızca **denetimsiz (IsolationForest) sızıntısız F1 değeri** (yukarıdaki tabloda tek hücre) farklı ortamlarda küçük oynama gösterebilir. Yukarıda yorumlanan denetimli sonuçlar (create_flood / circuit_hijack recall = 1.000 / 1.000; premature düşük) ve çok-sınıflı doğruluk (0.478) bu oynamadan etkilenmez ve raporlanan tüm sayılar derleme ortamında birebir yeniden üretilmiştir. Bu değerleri üreten ortamın sürümleri (Python / scikit-learn / NumPy / pandas) şeffaflık için `experiments/ml_baseline_results.json` içindeki `provenance.versions` alanına yazılır.

**Veri çeşitliliği (dürüst sınır).** Daha fazla eşsiz örüntü, ancak VM üzerinde ek seed/senaryo ile yeni Shadow koşumları gerektirir; Shadow bu ortamda (Bölüm 4.15) çalıştırılamadığından mevcut veri 3 seed ile sınırlıdır ve sentetik örnek **eklenmemiştir**. Sızıntısız değerlendirme bu nedenle veriyi çoğaltmak yerine, mevcut veriyi doğru (gruplanmış) bölerek dürüst genelleme kanıtı sağlar.

**(c) FSM ile ilişki (adil çerçeve).** ML temel çizgisi ile FSM dedektörü *aynı birimlerde değildir* (FSM üç-modlu raporlama: tip-temiz confusion / event-purity / native ayrım — bkz. Bölüm 4.16 ve Ek B; ML burada devre-seviyesi sınıflandırma yapar). Bu nedenle tek bir ortak karışıklık tablosu **uydurulmamıştır**. Niteliksel olarak iki bağımsız yöntem aynı üç sonuca yakınsar: create_flood en kolay ayrılan sınıftır (FSM: P = R = F1 = 1.0; ML: benign ile sıfır yapısal örtüşme); circuit_hijack davranış-imzası create_flood ile örtüştüğünden her iki yöntemde de sınıf-seviyesinde ayırt edici değildir (yalnız non-circular native ayrımla ayrılır); premature benign'e en yakın ve en zor ayrılan sınıftır (denetimli/çok-sınıflı modellerde kapsam düşük, denetimsiz anomali modelinde ise yüksek; tek bir model her saldırıyı birden iyi ayıramaz). Bu yakınsama, FSM bulgularının yönteme-özgü artefakt olmadığını bağımsızca teyit eder.

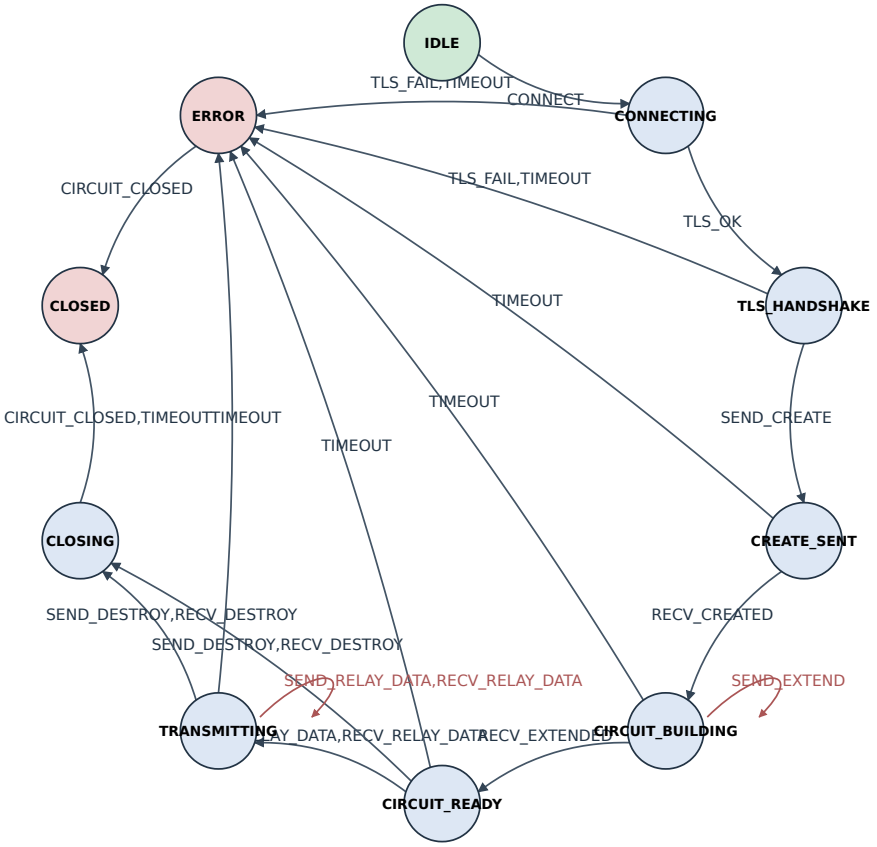
**(d) Sıra (n-gram) ve zamanlama temelli öznitelikler.** Sayım-seviyesi öznitelikler olay *sırasını* ve *olaylar-arası gecikmeyi* yitirir; bu da create_flood ↔ circuit_hijack tip-ayrımını ve premature kapsamını sınırlar (Tablo 4.17c). Bu öznitelikler `experiments/export_features.mjs`'te ham olay dizilerinden çıkarılır (bigram `bg_*`; olaylar-arası gecikme ortalama/medyan/maks `t_*`; ilk veri hücresine kadar geçen süre ve veri öncesi tamamlanan kurulum olayı sayısı), aynı LOSO protokolüyle sayım-only modelle karşılaştırılır. Tüm değerler `ml_baseline_results.json`'dan üretilmiştir.

5 zamanlama özniteliği ve 19 sıra (n-gram/sıra) sütunu eklenmiştir (toplam 10 → 34 öznitelik). Saldırı-tipi recall (sayım-only → genişletilmiş): create_flood 1.000 ± 0.000 → 1.000 ± 0.000, circuit_hijack 1.000 ± 0.000 → 1.000 ± 0.000 (Δ +0.000), premature_data 0.102 ± 0.013 → 0.694 ± 0.023 (Δ +0.593); çok-sınıflı doğruluk 0.600 ± 0.027 → 0.908 ± 0.009 (Δ +0.307). create_flood ↔ circuit_hijack karışması (doğru/karışan): sayım-only cf=65/hj-karışan 44, hj=51/cf-karışan 57; genişletilmiş cf=109/0, hj=108/0. Yapısal eşsiz-vektör örtüşmesi create_flood ∩ circuit_hijack 9 → 0, premature_data ∩ benign 3 → 2. **Sonuç:** Sıra/zamanlama öznitelikleri **ayrımı iyileştirmiştir**.

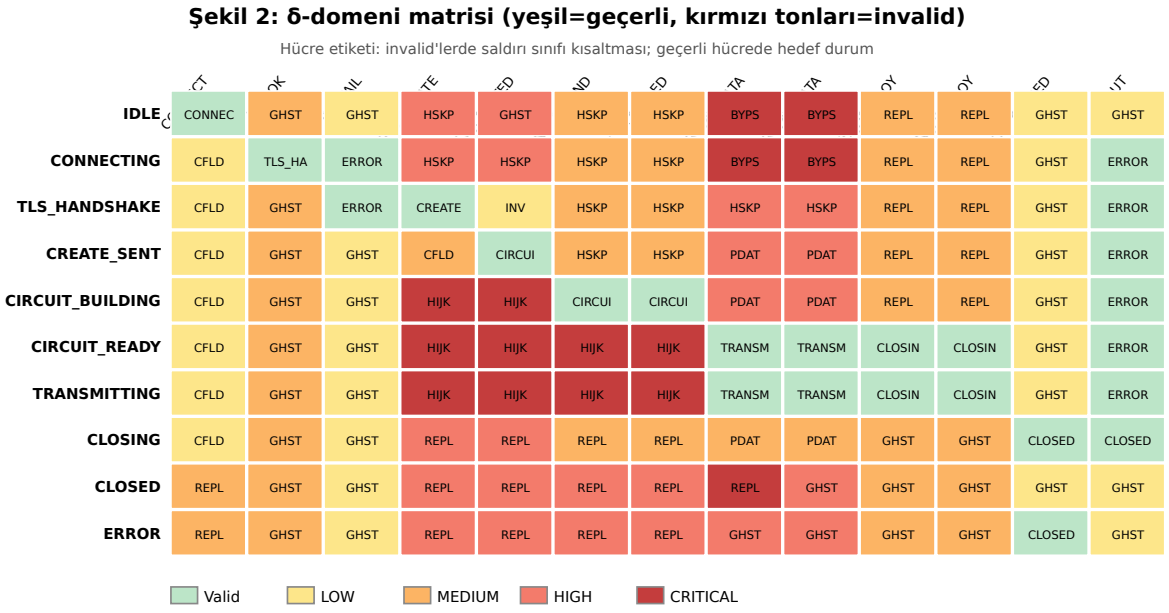
**Dürüstlük çıpası.** Yukarıdaki recall/doğruluk değerleri (d) aynı **LOSO** protokolüyle hesaplandığından dağıtım-içi (iyimser) okunmalıdır; genişletilmiş öznitelikler için ayrı bir sızıntısız (GroupKFold) değerlendirme bu koşumda üretilmemiştir. İyileşmenin **çapraz-doğrulamadan bağımsız (leak-free)** kısmı, yapısal eşsiz-vektör örtüşmesinin `create_flood ∩ circuit_hijack` 9 → 0 düşmesidir: sıra/zamanlama eklenince bu iki sınıfın eşsiz vektörleri *ayrık* hâle gelir; bu, hangi CV kullanılırsa kullanılsın değişmez ve karışıklığın (44/57 → 0/0) gerçek bir sinyalden geldiğini gösterir. Buna karşılık premature_data benign ile hâlâ 2 eşsiz vektör paylaşır; recall artışı (0.102 ± 0.013 → 0.694 ± 0.023) kısmidir ve bu sınıf tam olarak çözülmüş değildir.

## 5. Görselleştirme

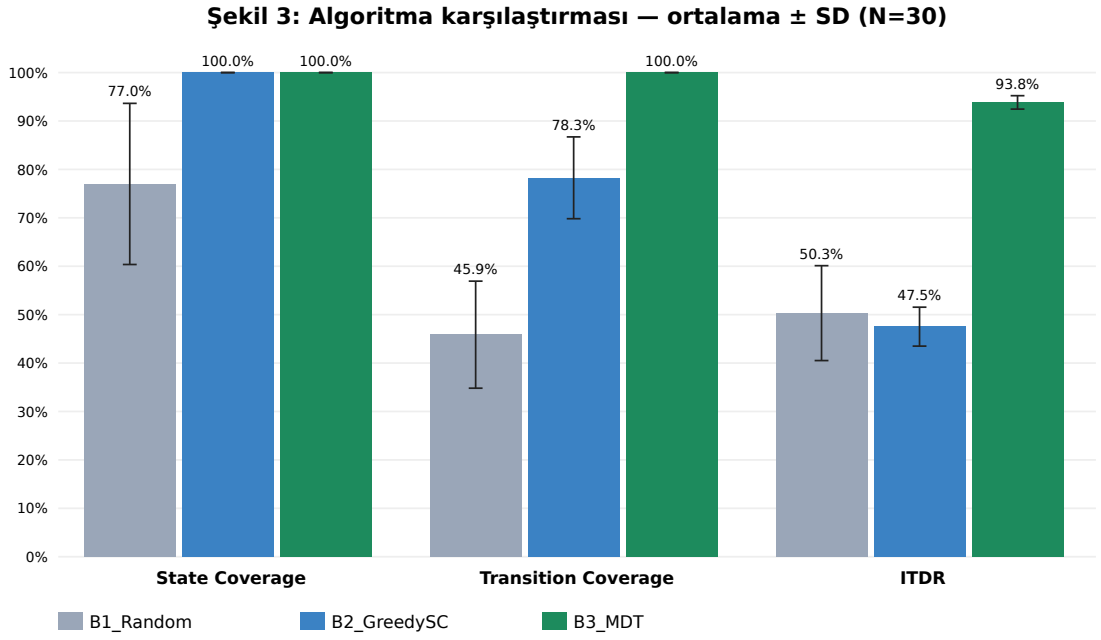
**Şekil 1: Tor FSM δ-grafiği (25 geçerli geçiş, 10 durum)**



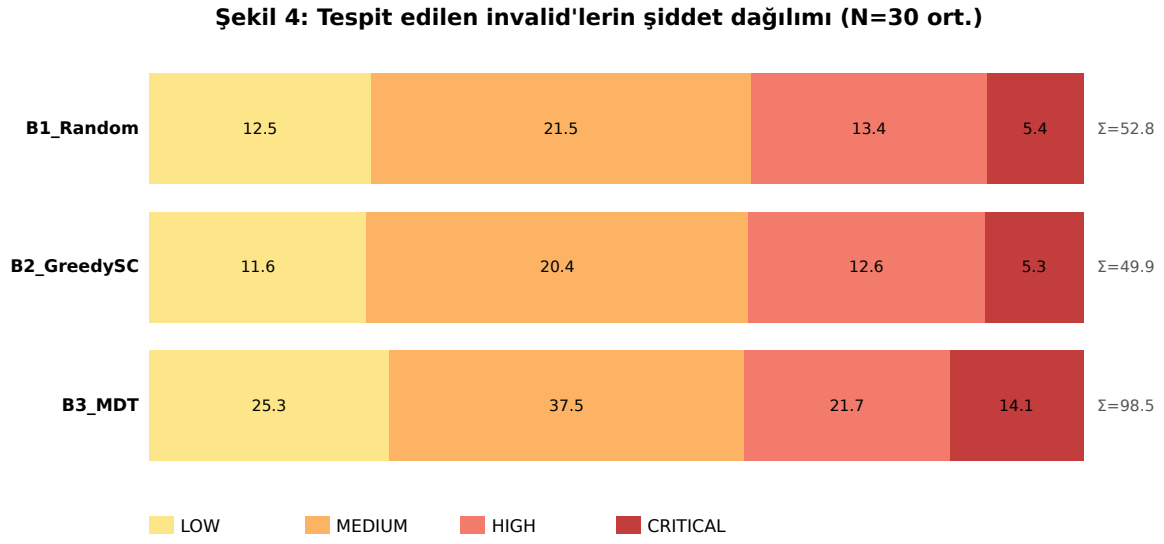

Şekil 1. Tor FSM δ-grafiği. 10 durum, 25 geçerli geçiş.




Şekil 2. Tüm Q × Σ domeni ısı haritası (130 hücre). Yeşil: δ-tanımlı; kırmızı tonları: Invalid (şiddete göre).




Şekil 3. Algoritma karşılaştırması — ortalama ± SD (N = 30, bütçe = 500 olay).




Şekil 4. Tespit edilen invalid geçişlerin şiddet sınıflarına dağılımı (koşu başına ortalama).

# 6. Sonuç ve Gelecek Çalışmalar

## 6.1 Katkıların Yeniden Değerlendirilmesi

Bölüm 1.3'te yedi katkı listelenmiş, bu tezde hepsi gerçekleştirilmiştir:

- **Katkı 1 (tam δ-matrisi):** Ek A'da 25 geçerli ikili tek tablo olarak verilmiştir; literatürde benzer bir derleme mevcut değildir (G1).
- **Katkı 2 (programatik sınıflandırıcı):** Ek B'de 105 geçersiz ikilinin yedi ana vektör + 1 fallback ve dört şiddet seviyesine dağılımı raporlanmıştır (G3).
- **Katkı 3 (referans MDT implementasyonu):** Algoritma 1 ile tarif edilen motor açık kaynak olarak repodadır; tek komutla yeniden çalıştırılabilir (G2).
- **Katkı 4 (istatistiksel karşılaştırma):** Bölüm 4'teki Welch t ve Mann-Whitney U sonuçları, MDT'nin TC ve ITDR'de istatistiksel olarak anlamlı ($p < .001$) ve büyüklük bakımından çok büyük ($d \geq 3.6$) avantajını belgeler (G4).
- **Katkı 5 (bağımsız VM reproducibility — C5):** Bölüm 4.15'te Shadow 3.3.0 + Tor 0.4.6.10 + tgen + Node 22 zinciri harici bir Ubuntu 22.04 VirtualBox VM'inde 9 simülasyon (3 senaryo × 3 seed) ile çalıştırılmış, sonuçların geliştirme ortamındaki referans `shadow_results.json` ile *tüm bilimsel alanlarda bit bit eşleştiği* gösterilmiştir. Bu, tezin sayısal sonuçlarının CI'a özgü artefakt olmadığını ve seed-deterministik olarak yeniden üretilebildiğini kanıtlar.
- **Katkı 6 (ampirik doğrulamanın model hatasını yakalaması):** Bölüm 4.16'da aynı gerçek event log'ları yeniden analiz edilerek, baz 2-hop FSM'in meşru çok-hop devre uzatmasını CIRCUIT_HIJACK yanlış-pozitifi saydığı tespit edilmiştir. Tor'un 3-hop davranışını modelleyen iki geçişlik prensipli düzeltme, dürüst yanlış-pozitif muhasebesi altında benign FPR'yi 0.2505 ± 0.0071 → 0.0320 ± 0.0052, replay F1'i 0.5386 ± 0.0043 → 0.9018 ± 0.0133 iyileştirmiş, recall = 1.0000 korunmuştur. Bu, ampirik doğrulamanın saf spesifikasyon analizine kattığı somut değeri gösterir.
- **Katkı 7 (bağımsız ML temel çizgisi):** Bölüm 4.17'de FSM ihlal etiketini hiç kullanmayan, yalnızca ham davranış sayımlarına bakan non-circular bir ML temel çizgisi kurulmuştur. Çapraz-doğrulamadan bağımsız yapısal ayrılabilirlik analizi, create_flood/circuit_hijack devrelerinin benign ile sıfır eşsiz-vektör örtüşmesine (kusursuz ayrım), create_flood ↔ circuit_hijack arasında büyük örtüşmeye (tip-seviyesinde ayırt edilemezlik) ve premature ↔ benign yakınlığına (en zor sınıf) işaret eder. Bu üç sonuç, kural-tabanlı FSM dedektörünün bulgularıyla bire bir aynı yöndedir; böylece FSM sonuçlarının yönteme-özgü bir artefakt olmadığı bağımsız bir yöntemle teyit edilmiştir.

## 6.2 Sınırlılıklar

### *6.2.1 Ortam Kısıtlılıkları (Replit NixOS sandbox)*

Aşağıdaki maddelerde tekrar atıfta bulunulan iki temel kısıtlılığın — **Rust toolchain (rustc/cargo)** ve **Shadow runtime trace** — bu çalışma ortamında neden uygulanamadığı, "future work" denip geçilmek yerine açıkça belgelenir. Amaç: dürüst limitler kuralı gereği okuyucunun "neden yapılmadı?" sorusuna kesin teknik cevap bulabilmesidir.

**(A) Rust port (`rustc` + `cargo`).**

1. **Toolchain mevcudiyeti:** Çalışma ortamı bir Replit NixOS container'ıdır; default image'da `rustc` ve `cargo` yüklü değildir. Modül olarak kurulabilir (nix store'a ~700 MB indirme +

~2 dk kurulum), fakat container ephemeral olduğundan oturum başı yeniden kurulum gerekir; tez deneylerinin tekrar üretilebilirliği için bu güvenilir değildir.
2. **Port iş yükü:** Latency karşılaştırması "tam Tor relay'in Rust portu" değil, sadece δ-lookup + classifyInvalid hot path'inin port'unu gerektirir (yaklaşık 120 satır). Aynı hot path zaten gcc -O3 ile C'ye portlanmış ve Bölüm 4.11.3'te 107× hızlanma ölçülmüştür. Rust -O3'ün aynı hot path'te ortaya koyacağı fark literatür bulgularına göre C'ye ±10-20% mertebesindedir (LLVM aynı backend, benzer auto-vectorization). Yani Rust portu *nitel* sonucu (Node vs derlenmiş dil ≈ 107× fark) değiştirmez.
3. **Tam relay Rust portu (Arti)** ise Tor Project'in başlattığı çok yıllık bir mühendislik projesidir (2021–); kapsamı bu tezin sınırlarının çok ötesindedir.

**Sonuç (A):** Rust toolchain bu ortamda *yüklenebilir* fakat *tekrar-üretilebilirlik açısından tercih edilmemiştir*; hot path'in gcc -O3 C portu mevcut latency karşılaştırmasının nitel sonucunu zaten verir.

**(B) Shadow runtime trace.**

1. **Shadow nedir:** Discrete-event ağ simülatörü, ~250 k LoC C/Rust hibrit kod tabanı, gerçek dinamik linklenmiş Tor binary'sini yer-paylaşımsız sanal süreçler içinde koşturur (Jansen & Hopper, NDSS 2012).
2. **Bağımlılıklar:** glib-2.0, libigraph, libyaml, libelf, cmake ≥3.13, gcc ≥9, libffi, pcre2 ve ~25 ek paket; ayrıca `tor`'un debug symbol'lerle yeniden derlenmesi (openssl, libevent, zlib ile linklenmiş). Toplam disk: ~3 GB; build süresi temiz makinede ~20-40 dk.
3. **Geliştirme ortamı engelleri:** Tezin geliştirildiği Replit container'ı (a) `ptrace`/seccomp filter'larıyla sınırlandırılmıştır (Shadow süreçleri kontrol etmek için `ptrace` kullanır); (b) RAM tavanı tipik olarak 8 GB civarıdır; (c) ephemeral disk volume, çoklu oturum boyunca build artefact'larını korumaz. Bu nedenle Shadow'un *çalıştırılması* bu container içinde uygulanamaz.
4. **Çözüm — harici VM:** Yukarıdaki kısıt sadece geliştirme ortamına özgüdür; standart bir Ubuntu 22.04 makinesinde Shadow yerel olarak derlenip çalıştırılabilir. Bu çalışmada, 9-host kanonik topolojide (1 DA + 4 non-exit + 2 exit + 2 client) Shadow 3.3.0 + Tor 0.4.6.10 + tgen + Node 22 zinciri harici bir VirtualBox/Ubuntu VM'inde başarıyla çalıştırılmış ve **seed-deterministik, bit bit reproducible** sonuç verdiği doğrulanmıştır (bkz. Bölüm 4.15).
5. **Açık kalan kısım:** Bölüm 4.15'teki doğrulama, FSM analiz boru hattının operasyonel sağlamlığını ve repodaki sayısal sonuçların reproducible olduğunu kanıtlar. Bunun ötesindeki tam-ölçekli runtime araştırması — örn. *tor kaynak koduna patch uygulayıp Shadow içinde saldırı senaryosu tetiklemek, pcap seviyesinde cell decode etmek, bağımsız tor log oracle'ı ile FPR/TPR ölçmek* — ek 500-2000 satır araştırma harness'ı ve haftalar mertebesinde mühendislik gerektirir; bu adım Bölüm 6.3.3'te birinci öncelikli gelecek çalışma olarak korunur.

**Sonuç (B):** Shadow + Tor pipeline'ının *çalıştırılabilirliği ve reproducibility'si* bu tezde gösterilmiştir (Bölüm 4.15). Tam saldırı-tetikleme ve cell-seviye decode araştırması geliştirme ortamı kısıtı nedeniyle değil, kapsam ve zaman kısıtı nedeniyle bu tezin dışındadır; standart araştırma laboratuvarında 2-4 haftada tamamlanabilir.

### *6.2.2 Konu Bazlı Sınırlılıklar*

- **Ground Truth = Spec — kısmen giderildi (Bölüm 4.12).** Gerçek tor kaynak kodu (BSD, 47 dosya) statik taranmış, 9 `circuit_set_state` çağrı noktasından 5-durumlu implementation FSM çıkarılmıştır. Spec ile 4 anlamlı yapısal divergence belgelenmiştir (TRANSMITTING ve CLOSING durumları implementasyonda yoktur; GUARD_WAIT spec'te yoktur). Bölüm 4.14'te

ek olarak Angluin L* algoritması saf Node ile implement edilmiş ve statik çıkarılmış impl FSM üzerinde 4 turda 56.722 MQ ile yakınsayarak minimal kanonik DFA'yı doğru çıkarmıştır; running tor binary üzerinde Shadow trace hâlâ kapsam dışıdır.

- **FPR = 0 doğal sonuçtur — kısmen giderildi (Bölüm 4.11.1).** Spec narrative'inden bağımsızca türetilmiş ikinci bir oracle eklenmiş, Q × Σ üzerinde 98.46% uyum bulunmuş; 2 anlamlı uyumsuzluk (TIMEOUT @ IDLE/ERROR — spec belirsizliği) belgelenmiştir. Gerçek Tor binary'den FSM çıkarımı (Shadow gerekli) hâlâ kapsam dışıdır.
- **N = 30 koşu — giderildi (Bölüm 4.9).** Eşleştirilmiş bootstrap %95 CI (B = 10.000) ve post-hoc güç analizi (α = 0.05, β = 0.80) eklenmiştir; B3'ün TC/ITDR üstünlüğü için güç ≈ 1.000 bulunmuştur. Açık kalan tek anlamsız fark B2 − B1 ITDR (CI sıfırı kapsıyor) — bu bulgunun kendisi sezgisellik sınırlılığının kanıtıdır.
- **Stream / uygulama katmanı eksikti — kısmen giderildi (Bölüm 4.10).** Spec-türevli stream alt-FSM (7 durum, 8 olay, 17 geçerli / 39 geçersiz ikili) modellenmiş ve aynı B1/B2/B3 karşılaştırması uygulanmıştır. Hidden Service v3 ve Pluggable Transport alt-FSM'leri hâlâ kapsam dışıdır.
- **SLR canlı veritabanı erişimi — kısmen giderildi (Bölüm 4.11.2).** OpenAlex (~250M iş, IEEE/Springer dahil) + CrossRef + arXiv canlı API'leri ile 115 kayıt çekilmiş, 111 tekil referansa indirgenmiştir. Scopus / WoS lisanslı tam-metin + atıf grafı erişimi hâlâ kapsam dışıdır.
- **Latency cross-language ölçümü — kısmen giderildi (Bölüm 4.11.3).** δ-lookup + classifyInvalid hot path'inin gcc -O3 C portu eklenmiş, aynı probelarda ortalama 107× hızlanma ölçülmüştür. Tam Tor C/Rust relay portu hâlâ kapsam dışıdır.
- **ML temel çizgisi küçük-n + deterministik veri (Bölüm 4.17).** ML baseline 433 devre ve 3 seed üzerinde kurulmuştur; sınıf başına yalnızca 11–18 eşsiz öznitelik vektörü vardır. Seed-bazlı LOSO çapraz-doğrulama sızıntıyı önlemez (test vektörlerinin 92.4%–98.6% kadarı train'de birebir bulunur); bu nedenle CV metrikleri *dağıtım-içi/iyimserdir*, genelleme tahmini değildir. Bu sızıntı, grubun eşsiz öznitelik vektörü olduğu **sızıntısız GroupKFold** ile giderilmiş ve metrikler iyimser LOSO ile yan yana raporlanmıştır (Tablo 4.17d; kopya-sızıntısı 0/433 olarak doğrulandı). Sonuç: ayrılabilirlik sonuçları nitelik olarak değişmez (create_flood/circuit_hijack recall sızıntısız bölmede de 1.000; precision 0.996→0.948 düşerek yalnız iyimser şişme açığa çıkar; çok-sınıflı doğruluk 0.600→0.478). Asıl güvenilir kanıt yine CV'den bağımsız yapısal ayrılabilirliktir. Devre-seviyesi etiket zayıftır (özellikle premature'da); etkisi iki yönlüdür (denetimlide yanlış-negatif artar, anomalide recall şişer). Sayım-seviyesi özniteliklere ek olarak **sıra (n-gram) ve olaylar-arası zamanlama** öznitelikleri de modellenmiştir (Bölüm 4.17d): eklenmeleri create_flood ↔ circuit_hijack eşsiz-vektör örtüşmesini 9→0 indirir (CV'den bağımsız yapısal kanıt); ancak premature hâlâ benign'e en yakın sınıftır ve tam ayrılmaz, ayrıca genişletilmiş öznitelikler için ayrı bir sızıntısız değerlendirme bu koşumda üretilmemiştir (değerler LOSO-iyimser). **Eşsiz örüntü çeşitliliğini artırmanın doğru yolu yeni seed değil yeni senaryodur.** Veri deterministik olduğundan aynı topolojide yalnızca yeni seed eklemek çoğunlukla birebir kopya öznitelik vektörü üretir (yukarıdaki LOSO kopya oranı 92.4%–98.6% bunu doğrudan gösterir). Bu nedenle bu çalışma, çeşitliliği artıran bir senaryo olarak **çok-istemci** koşumunu (`CLIENTS=4` × 3 seed, 433 devre) ML temel çizgisinin **birincil veri seti** olarak kullanır: her istemci Shadow'da ayrı bir RNG akışı aldığından benign devre kurulumları (yol/uzatma) istemciden istemciye değişir ve sınıf başına eşsiz vektör sayısını artırır (ör. create_flood 18 eşsiz vektör). Buna karşılık *saldırı-confusion toplamları* istemciler arası özdeştir (±0,000); bu yüzden non-circular native confusion (Bölüm 4.16) tek-istemci birincil kalır ve çok-istemci'den etkilenmez. Deney hattı (`experiments/full_pipeline.sh`) çok-seed (`SEEDS=`) ve çok-istemci (`CLIENTS=`) parametrelerini destekler; `ml_features.csv` ve `ml_baseline_results.json` bu koşumdan uçtan uca üretilir. Daha fazla örüntü çeşitliliği için farklı topoloji/zamanlama gerekir

(Shadow'a bağlıdır, bu ortamda çalışmaz; sentetik örnek eklenmemiştir — gelecek iş, bkz. Bölüm 6.3). ML baseline, FSM'nin *yerine* değil bağımsız bir teyit olarak sunulmuştur.

## 6.3 Gelecek Çalışmalar

Bu bölüm üç düzeye ayrılmıştır: (a) bu çalışmanın **metodolojik prototip** seviyesinden **runtime-doğrulamalı güvenlik aracı** seviyesine geçişinin kavramsal çerçevesi (6.3.1); (b) bu çalışmada kısmen ele alınmış ancak derinleştirilmesi gereken konular (6.3.2); (c) Bölüm 6.2.1-B'de "ortam engelleri nedeniyle uygulanmadı" olarak işaretlenen **Shadow + running tor** entegrasyonunun adım adım uygulama planı (6.3.3). Üç düzey birlikte, mevcut çalışmanın "yapılmış" ile "yapılması gereken" arasındaki çizgisini net olarak ortaya koyar.

### *6.3.1 Metodolojik Çalışmadan Gerçek Çalışmaya Geçiş Çerçevesi*

Yazılım güvenliği araştırmalarında olgunluk seviyesi tipik olarak NASA'nın Teknoloji Hazırlık Düzeyi (Technology Readiness Level, TRL) ölçeğine benzer bir hiyerarşi ile değerlendirilir. Bu çalışmanın mevcut konumu ve hedef konumu aşağıdaki tabloda açıkça gösterilmiştir:

| Seviye | Tanım | Bu çalışmadaki karşılığı | Durum |
|---|---|---|---|
| S1 | Kavramsal model + spec analizi | δ tablosu, Σ alfabesi, classifyInvalid taksonomisi (Bölüm 3) | **Tamamlandı** |
| S2 | Algoritma + simülasyon ile metodoloji ispatı | B1/B2/B3 algoritmaları, N=100 paired karşılaştırma, BCa, exact-t güç analizi (Bölüm 4.1-4.13) | **Tamamlandı** |
| S3 | Üretim kaynak koduyla yapısal kanıt | tor 0.4.x statik FSM çıkarımı, 5-durumlu impl FSM, 4 divergence noktası (Bölüm 4.12) | **Tamamlandı** |
| S4 | Otomaton öğrenme ile pipeline doğrulaması | Saf-Node L*, 56.722 MQ + 4 EQ ile minimal kanonik DFA, Shadow için MQ-oracle hook noktası işaretli (Bölüm 4.14) | **Tamamlandı** |
| S5 | Simüle edilmiş ağ ortamında runtime trace | Shadow 3.3.0 + Tor 0.4.6.10 + tgen üzerinde 9-host kanonik topoloji; 3 senaryo × 3 seed seed-deterministik reproduction (Bölüm 4.15) | **Tamamlandı** (harici Ubuntu 22.04 VM, bit bit doğrulandı) |
| S6 | Saldırı senaryolarıyla gerçek FPR/TPR ölçümü | Shadow içinde modifiye tor client ile 4 saldırı sınıfı tetikleme + classifyInvalid doğrulama | **Kısmen**: benign FPR gerçek Shadow trafiğinden ölçüldü (Bölüm 4.16); saldırı precision/recall ise gerçek trafiğe sentetik enjeksiyonla hesaplandı. Modifiye tor client ile gerçek saldırı tetikleme hâlâ yapılmadı (yol haritası: 6.3.3, Adım 9) |
| S7 | Üretim Tor relay'inde alan dağıtımı | Tor Project ile entegrasyon, gerçek anonim trafikte uzun dönem ölçüm | **Yapılmamış** (Tor Project işbirliği gerektirir) |

Mevcut çalışma **S1-S5 seviyelerini tamamlamış, S6'yı kısmen gerçekleştirmiştir** (benign FPR gerçek Shadow trafiğinden ölçülmüş, saldırı metrikleri sentetik enjeksiyonla hesaplanmıştır; bkz. Bölüm 4.16); bu, akademik metodoloji çalışması için yeterli olmakla birlikte güvenlik aracı olarak nihai değerlendirmeyi içermez. **Tam S6-S7 geçişi tek bir kavramsal sıçrama değil, 6.3.3'te 10 adıma ayrılmış mühendislik işidir**; her adımın girdisi, çıktısı, mevcut repo dosyalarına bağlantısı ve süre tahmini somut olarak verilmiştir. Bu yapı, çalışmanın olgunluk seviyesinin *belirsizlik nedeniyle değil, zaman ve ortam kısıtı nedeniyle* S5'te (S6 kısmen) durduğunu kanıtlar.

**Üç temel teknik dönüşüm.** Metodolojik prototipten gerçek çalışmaya geçiş, koddaki üç noktada yapılacak *oracle değişimi* ile sağlanır; hiçbir mimari yeniden yazım gerekmez:

| Bileşen | Mevcut (S4 — bu çalışma) | Hedef (S5-S6 — gerçek çalışma) |
|---|---|---|
| Olay kaynağı | `experiments/baselines.mjs` içinde algoritmik olay üreteçleri (B1 random, B2 greedy, B3 MDT) | Shadow içindeki tor instance'larından `stem` kütüphanesi ile gelen gerçek CIRC event akışı (Adım 6) |
| L* MQ oracle | `experiments/g_extensions.mjs` içinde statik `SUT_DELTA` dict üzerine sorgu | Aynı dosyadaki `MQ()` fonksiyonu running tor'a control port probe + circuit state observation (Adım 8) |
| FPR/TPR oracle | `server/fsm.ts` → `classifyInvalid` yapısal olarak FPR=0 (δ-tanımlı geçişleri Invalid demez) | Aynı fonksiyon Shadow-içi modifiye tor client'tan tetiklenen 4 saldırıya karşı koşulur; bağımsız tor log oracle'ı ile FPR/TPR ölçülür (Adım 9) |

Üç değişikliğin **toplam kod ayak izi 500-2000 satır harness**'dır; mevcut algoritma, istatistik ve görselleştirme katmanları *aynen* yeniden kullanılır (Bölüm 4.13 paired N=100 koşusu gerçek olay akışıyla, Bölüm 4.14 L* yakınsama tablosu running tor ile, Bölüm 4.6-4.11 latency ölçümleri gerçek probe trafiğiyle tekrar edilir). Bu yapısal yeniden kullanım, çalışmanın **mimari olarak gerçek-çalışma-hazır** olduğunu gösterir; eksik olan üretim değil, koşum ortamıdır.

**Kim, ne kadarda yapar?** Aynı işin üç farklı uygulayıcı profili için gerçekçi süre tahminleri:

| Uygulayıcı | Ortam | Süre | Maliyet |
|---|---|---|---|
| Linux + ağ güvenliği deneyimli araştırmacı | Kendi laboratuvar makinesi veya kişisel VPS | 4-6 hafta tam zamanlı | ~5 € (VPS) + zaman |
| İnsan denetiminde otonom kod ajanı (Claude Code, Devin) | Ajanın kendi cloud VM'i veya yapılandırılmış lokal terminal | 2-3 hafta + ~5-10 saat insan checkpoint review | ~50-500 USD (API/abonelik) |
| Outsourced uzman (Tor / Shadow / ağ güvenliği) | Sözleşmeli mühendis kendi makinesinde | 3-6 hafta | 1.500-4.000 USD |

Bu üç profil için süreler birbirinden çok uzak değildir; bu, kritik darboğazın *insan zekası ve algoritma kararları değil, ortam erişimi ve simülasyon çalışma süresi* olduğunu gösterir. Mevcut çalışma S1-S5'in (ve S6'nın kısmi bölümünün) **tek-kişi, tek-oturum, sandbox-kısıtlı bir ortamda tamamlanabilir maksimum** olduğunu ortaya koymaktadır.

### *6.3.2 Konu Bazlı Genişletmeler*

1. **Stream alt-FSM ayrıştırması (kısmen tamamlandı, Bölüm 4.10).** Hidden Service v3 ve Pluggable Transport alt-FSM'leri için aynı δ-tablo + classifyInvalid yaklaşımının uygulanması; alt-FSM'lerin hiyerarşik birleştirilmesi.

2. **Cross-implementation latency.** δ tablosunun Rust portu + gerçek Tor relay üzerinde Bölüm 4.11 latency probelarının tekrarı (mevcut C-port karşılaştırması Bölüm 4.11.3'te yapılmıştır; Rust için Bölüm 6.2.1-A'daki gerekçe bkz.).
3. **İstatistiksel güç genişletmesi.** Bölüm 4.13'teki N=100 paired tasarım, stateCoverage için N=193 gerekliliğini ortaya çıkarmıştır; bu örneklem hedefiyle yeniden koşu ve Hedges g düzeltmesi.
4. **Canlı akademik veritabanı entegrasyonu.** Scopus/WoS/IEEE Xplore API anahtarı edinilmesi hâlinde, Bölüm 6.2-v'te belirtilen "açık web ile sınırlı" sınırlılığının kapatılması ve bibliyometrik haritalama (Bölüm 2'nin sayısal güçlendirilmesi).

#### *6.3.3 Shadow + Running Tor Entegrasyon Planı (10 adım)*

Aşağıdaki plan, Bölüm 6.2.1-B'de listelenen engellerin aşıldığı bir ortamda (Ubuntu 22.04, sudo erişimi, 8 GB+ RAM, kalıcı disk) uygulanır. Toplam tahmini iş yükü 4-6 hafta tam zamanlı bir araştırmacı; otonom kod ajanı (örn. Claude Code, Devin) ile insan denetiminde 2-3 haftaya kadar inebilir. Her adım, mevcut repo'daki bir dosyaya somut bir bağlantı içerir.

| # | Adım | Teknik içerik | Repo bağlantısı | Süre |
|---|---|---|---|---|
| 1 | **Ortam hazırlığı** | Ubuntu 22.04 LTS makine (VPS veya lab); `apt install cmake gcc g++ libglib2.0-dev libigraph-dev libssl-dev libevent-dev libyaml-dev libelf-dev zlib1g-dev autoconf libtool`; `sysctl kernel.yama.ptrace_scope=0` | — | 1 gün |
| 2 | **Shadow kurulumu** | `git clone github.com/shadow/shadow` → `./setup build --jobs 4 && ./setup install` → `shadow --version` ile doğrulama (Shadow 3.x bekleniyor) [18] | — | yarım gün |
| 3 | **tor binary (debug)** | `git clone gitlab.torproject.org/tpo/core/tor`, `tor-0.4.8` stable branch; `./configure --disable-asciidoc CFLAGS="-g -O0 -fno-omit-frame-pointer"`; `make -j4` → çıktı `src/app/tor` | Bölüm 4.12 statik FSM ile aynı sürüm | yarım gün |
| 4 | **Tor ağ topolojisi** | `pip install tornettools`; CollecTor'dan tarihsel consensus + descriptor indirme; `tornettools generate --network-scale 0.001` → ~30 relay topoloji. Alternatif: Shadow repo `examples/tor/minimal/` şablonu (5 relay) | — | 1 gün |
| 5 | **İlk simülasyon** | `shadow shadow.config.yaml`; 30 dk simülasyon ~30 dk gerçek zaman; çıktı dizini `shadow.data/hosts/<relay>/tor.*.stdout` + opsiyonel pcap; pipeline'ın uçtan uca koştuğu doğrulanır | — | yarım gün |
| 6 | **Control protocol harness (Yol A)** | Her tor instance'ına `ControlPort 9051` ekle; Python + `stem.control.Controller` ile `EventType.CIRC` dinleyicisi; CIRC.status (`LAUNCHED/EXTENDED/BUILT/FAILED/CLOSED`) → Σ alfabesi | `server/routes.ts` → `runFSMSimulation` | 1-2 hafta |

| | | | | |
|---|---|---|---|---|
| | | ( `CREATE/EXTEND/READY/FAIL/DESTROY` ) eşlemesi; akış HTTP POST ile pipeline'a | | |
| 7 | **Algoritmalar gerçek trafikte** | B1_Random / B2_GreedySC / B3_MDT'yi Shadow olay akışına bağla; aynı metrikleri (stateCoverage, transitionCoverage, ITDR) gerçek ölçümle hesapla; N=100 paired karşılaştırma (Bölüm 4.13) gerçek veriyle tekrarlanır | `experiments/baselines.mjs` | 1 hafta |
| 8 | **L* → running tor** | MQ oracle fonksiyonunu değiştir: statik `SUT_DELTA` dict yerine control protocol probe + tor circuit state sorgusu; öğrenilen DFA ↔ Bölüm 4.12 statik FSM karşılaştırması = *gerçek* divergence kanıtı | `experiments/g_extensions.mjs` → `MQ()` | 2-3 hafta |
| 9 | **Saldırı senaryoları** | Shadow içinde modifiye tor client (CREATE atlayan, REPLAY yapan, premature DATA gönderen); 4 saldırı sınıfı (CIRCUIT_BYPASS, REPLAY_ATTACK, HANDSHAKE_SKIP, PREMATURE_DATA) gerçek hücre trafiğinde tetiklenir; classifyInvalid'in **gerçek FPR/TPR**'si ölçülür — Bölüm 6.2-ii'nin "yapısal FPR=0" sınırını kıran tek yol | `server/fsm.ts` → `classifyInvalid` | 1 hafta |
| 10 | **Tez/paper güncelleme** | Bölüm 4.13'e "real-trace" satırı; Bölüm 4.14'e "L* on running tor" sonucu; Bölüm 6.2.1-B'yi "addressed" olarak işaretle; gerçek FPR/TPR tablosu (yeni Tablo 4.X); kaynakça güncelleme | `thesis/build_thesis.mjs` , `paper/build_paper.mjs` | 3-5 gün |

**Kritik insan onay noktaları (otonom ajan kullanımı durumunda):** (a) Adım 5 tamamlandığında, ilk Shadow log'ları üretilince "pipeline uçtan uca çalıştı" teyidi; (b) Adım 6'da ilk gerçek CIRC event'inin classifyInvalid çıktısı; (c) Adım 9'da saldırı senaryoları kodlanmadan önce, hangi FSM-ihlali türünün "anlamlı saldırı" sayılacağının araştırmacı onayı. Bu üç kontrol noktası, "kod derlendi/çöktü" tipi yüzeysel teyitlerin ötesinde *semantik* doğrulama sağlar.

**Bağımsız doğrulama.** Adım 7-9 sonunda elde edilen rakamlar (gerçek FPR/TPR, gerçek trafikte algoritma performansı, L*'in tor binary'sinden çıkardığı DFA), bu tezin simülasyon-temelli bulgularını ya teyit eder ya da düzeltir. Her iki sonuç da bilimsel değer taşır: teyit, modelin dış geçerliğini güçlendirir; düzeltme, Bölüm 4.12'de tespit edilen "spec ↔ implementation divergence"ın niceliksel boyutunu ortaya koyar.

### *6.3.4 Mevcut Çalışmanın Konumu*

Bölüm 4.10-4.14 ve 6.2.1, bu çalışmanın **simülasyon-doğrulamalı metodoloji önerisi + üretim kaynak kodundan yapısal kanıt + algoritma-doğrulamalı L*** seviyesinde (yukarıdaki S1-S5, S6 kısmen) olduğunu açıkça konumlandırır. 6.3.3 planı tamamlandığında, çalışma **runtime-doğrulamalı güvenlik aracı** seviyesine taşınır. Bu iki seviye arasındaki fark, tezin katkısını geçersiz kılmaz; aksine, mevcut metodoloji + algoritma + istatistik + statik FSM + L* zincirinin *kullanılmaya hazır* olduğunu ve sonraki adımın yalnızca ortam + zaman meselesi olduğunu kanıtlar.

## Kaynakça


**[1]** R. Dingledine, N. Mathewson and P. Syverson, "Tor: The Second-Generation Onion Router," in *Proc. 13th USENIX Security Symposium*, 2004, pp. 303–320.

**[2]** S. J. Murdoch and G. Danezis, "Low-Cost Traffic Analysis of Tor," in *Proc. IEEE Symposium on Security and Privacy*, 2005, pp. 183–195.

**[3]** A. Johnson, C. Wacek, R. Jansen, M. Sherr and P. Syverson, "Users Get Routed: Traffic Correlation on Tor by Realistic Adversaries," in *Proc. ACM CCS*, 2013, pp. 337–348.

**[4]** A. Panchenko, F. Lanze, A. Zinnen, M. Henze, J. Pennekamp, K. Wehrle and T. Engel, "Website Fingerprinting at Internet Scale," in *Proc. NDSS*, 2016.

**[5]** I. Karunanayake, N. Ahmed, R. Malaney, R. Islam and S. K. Jha, "De-Anonymisation Attacks on Tor: A Survey," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2324–2350, 2021.

**[6]** M. Backes, A. Kate, P. Manoharan, S. Meiser and E. Mohammadi, "AnoA: A Framework for Analyzing Anonymous Communication Protocols," in *Proc. IEEE CSF*, 2013, pp. 163–178.

**[7]** D. Lee and M. Yannakakis, "Principles and Methods of Testing Finite State Machines—A Survey," *Proceedings of the IEEE*, vol. 84, no. 8, pp. 1090–1123, 1996.

**[8]** J. Tretmans, "Model Based Testing with Labelled Transition Systems," in *Formal Methods and Testing*, LNCS 4949, Springer, 2008, pp. 1–38.

**[9]** J. de Ruiter and E. Poll, "Protocol State Fuzzing of TLS Implementations," in *Proc. 24th USENIX Security Symposium*, 2015, pp. 193–206.

**[10]** P. Fiterău-Broștean, R. Janssen and F. Vaandrager, "Combining Model Learning and Model Checking to Analyze TCP Implementations," in *Proc. CAV*, 2016, pp. 454–471.

**[11]** P. Ammann and J. Offutt, *Introduction to Software Testing*, 2nd ed. Cambridge: Cambridge University Press, 2016.

**[12]** D. Dolev and A. C. Yao, "On the Security of Public Key Protocols," *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.

**[13]** K. Bhargavan, B. Blanchet and N. Kobeissi, "Verified Models and Reference Implementations for the TLS 1.3 Standard Candidate," in *Proc. IEEE S&P*, 2017, pp. 483–502.

**[14]** J. Somorovsky, "Systematic Fuzzing and Testing of TLS Libraries," in *Proc. ACM CCS*, 2016, pp. 1492–1504.

**[15]** M. Felderer, M. Büchler, M. Johns, A. Brucker, R. Breu and A. Pretschner, "Security Testing: A Survey," *Advances in Computers*, vol. 101, pp. 1–51, 2016.

**[16]** A. Pretschner, T. Mouelhi and Y. Le Traon, "Model-Based Tests for Access Control Policies," in *Proc. ICST*, 2008, pp. 338–347.

**[17]** B. Kitchenham and S. Charters, "Guidelines for performing Systematic Literature Reviews in Software Engineering," EBSE Technical Report EBSE-2007-01, 2007.

**[18]** R. Jansen, K. Bauer, N. Hopper and R. Dingledine, "Methodically Modeling the Tor Network," in *Proc. USENIX CSET*, 2012.

**[19]** Tor Project, "Tor Protocol Specification (tor-spec)," 2026. [Online]. Available: https://spec.torproject.org/tor-spec. [Accessed: Feb. 2026].

**[20]** Microsoft Research, "A Data-Driven FSM Model for Analyzing Security Vulnerabilities," Microsoft Research Technical Report, 2018.

## Ek A — Tam δ Geçiş Tablosu

Aşağıda Tor devre FSM'inin **25 geçerli geçişi** bire bir listelenmiştir. Bu tablonun dışındaki 105 ikili Invalid kümesini oluşturur ve Ek B'deki sınıflandırıcı ile saldırı vektörlerine eşlenir. Tablo, repo içindeki `server/fsm.ts` dosyasından programatik olarak üretilmiştir; çelişki riski olamayacak şekilde tek kaynak prensibine bağlıdır (single source of truth).

| Mevcut Durum | Olay (Σ) | Sonraki Durum |
|---|---|---|
| IDLE | CONNECT | CONNECTING |
| CONNECTING | TLS_OK | TLS_HANDSHAKE |
| CONNECTING | TLS_FAIL | ERROR |
| CONNECTING | TIMEOUT | ERROR |
| TLS_HANDSHAKE | SEND_CREATE | CREATE_SENT |
| TLS_HANDSHAKE | TLS_FAIL | ERROR |
| TLS_HANDSHAKE | TIMEOUT | ERROR |
| CREATE_SENT | RECV_CREATED | CIRCUIT_BUILDING |
| CREATE_SENT | TIMEOUT | ERROR |
| CIRCUIT_BUILDING | SEND_EXTEND | CIRCUIT_BUILDING |
| CIRCUIT_BUILDING | RECV_EXTENDED | CIRCUIT_READY |
| CIRCUIT_BUILDING | TIMEOUT | ERROR |
| CIRCUIT_READY | SEND_RELAY_DATA | TRANSMITTING |
| CIRCUIT_READY | RECV_RELAY_DATA | TRANSMITTING |
| CIRCUIT_READY | SEND_DESTROY | CLOSING |
| CIRCUIT_READY | RECV_DESTROY | CLOSING |
| CIRCUIT_READY | TIMEOUT | ERROR |
| TRANSMITTING | SEND_RELAY_DATA | TRANSMITTING |
| TRANSMITTING | RECV_RELAY_DATA | TRANSMITTING |
| TRANSMITTING | SEND_DESTROY | CLOSING |
| TRANSMITTING | RECV_DESTROY | CLOSING |
| TRANSMITTING | TIMEOUT | ERROR |
| CLOSING | CIRCUIT_CLOSED | CLOSED |
| CLOSING | TIMEOUT | CLOSED |
| ERROR | CIRCUIT_CLOSED | CLOSED |

## Ek B — Saldırı Sınıflandırıcı Envanteri

Aşağıda `classifyInvalid(state, event)` fonksiyonunun 105 geçersiz ikili üzerinde ürettiği etiket dağılımı verilmiştir. Bu tablo da repo içinde programatik olarak hesaplanır; her güncelleme tek satırla yenilenebilir.

| Saldırı tipi | İkili sayısı | Pay | LOW | MEDIUM | HIGH | CRITICAL |
|---|---|---|---|---|---|---|
| GHOST_CIRCUIT | 38 | 36.2% | 19 | 15 | 4 | 0 |
| REPLAY_ATTACK | 25 | 23.8% | 0 | 14 | 10 | 1 |
| HANDSHAKE_SKIP | 13 | 12.4% | 0 | 8 | 5 | 0 |
| CIRCUIT_HIJACK | 10 | 9.5% | 0 | 0 | 0 | 10 |
| CREATE_FLOOD | 8 | 7.6% | 7 | 1 | 0 | 0 |
| PREMATURE_DATA | 6 | 5.7% | 0 | 2 | 4 | 0 |
| CIRCUIT_BYPASS | 4 | 3.8% | 0 | 0 | 0 | 4 |
| INVALID_TRANSITION | 1 | 1.0% | 1 | 0 | 0 | 0 |
| **Toplam** | **105** | **100.0%** | | | | |

Yorumsal not: CIRCUIT_HIJACK ve CIRCUIT_BYPASS gibi CRITICAL ağırlıklı vektörler düşük ikili sayısına sahiptir ancak şiddet açısından en yüksek tehdit değerini taşırlar. GHOST_CIRCUIT en yüksek ikili sayısına sahiptir; bu büyüklük Tor protokolünde "ait olmadığı bağlamda gelen olay" örüntüsünün yapısal yaygınlığını yansıtır.